

```
In [2]: import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

4-1 선형 모델

선형 모델은 통계 분석과 머신러닝에서 공통으로 사용되는 모델입니다.

- 통계 분석에서는 변수 간의 관계를 파악하여

현상을 설명하고 분석하기 위해 사용됩니다.

- 머신러닝에서는 예측과 분류 문제의 해결과 성능 향상에 중점을 둡니다.

[Ex.1]

간단한 선형 회귀 데이터셋을 만들어 보고,
statsmodels 통계 라이브러리에서 제공하는 기능과
sklearn 머신러닝 라이브러리에서 제공하는 기능의 특징을 살펴봅니다.

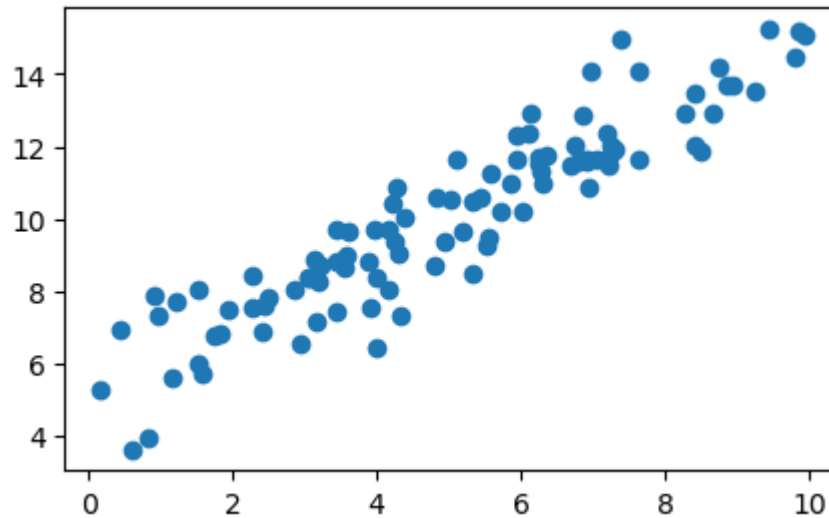
```
In [10]: Image('153.png')
```

Out[10]: $y = X + 5 + e, e \sim N(0, 1)$

관계를 지닌 데이터 포인트 100개를 만듭니다.

```
In [11]: # 모든 경우에 대해 동일한 데이터셋이
# 되도록 랜덤 시드를 설정합니다.
a, b = 1, 5
np.random.seed(123)
X = np.random.random(size=100) * 10
y = X * a + b + np.random.normal(size=100)

plt.figure(figsize=(5, 3))
plt.scatter(X, y)
plt.show()
```



```
In [12]: '''
코드는 파이썬의 `statsmodels` 라이브러리를 사용하여
선형 회귀 분석을 수행하는 것으로 보입니다.
여기서 `OLS`는 최소 제곱 회귀(Ordinary Least Squares Regression)
모델을 나타내며, 이 모델은 종속 변수와
하나 이상의 독립 변수 간의 관계를 설명하는 선형 모델입니다.
```

1. `lm = OLS(y, add_constant(X)).fit()`:
 여기서 `y`는 종속 변수를 나타내고,
`X`는 독립 변수를 나타냅니다.
`add_constant()` 함수는 독립 변수 행렬에
 상수항을 추가합니다.
 이는 회귀 모델에서 상수항이 있는 경우
 회귀 계수를 추정하는 데 도움이 됩니다.
 그런 다음 `OLS` 객체를 생성하고
`fit()` 메서드를 호출하여 모델을 데이터에 맞춥니다.
2. `lm.summary()`:
 마지막으로, 모델의 요약 통계를 출력합니다.
 이는 회귀 분석 결과를 요약하여 회귀 계수,
 모델 적합도 지표 및 다른 통계적 정보를 제공합니다.
 이를 통해 모델의 유효성과 예측 능력을 평가할 수 있습니다.

따라서 코드는 주어진 종속 변수와 독립 변수를 사용하여
 최소 제곱 회귀 모델을 적합하고, 그 결과를 요약하여 출력합니다.
 ...

```
from statsmodels.api import OLS, add_constant
# statsmodels.
lm = OLS(y, add_constant(X)).fit()
lm.summary()
```

Out[12]:

OLS Regression Results						
Dep. Variable:		y			R-squared:	0.856
Model:		OLS			Adj. R-squared:	0.855
Method:		Least Squares			F-statistic:	583.8
Date:		Sun, 19 May 2024			Prob (F-statistic):	4.57e-43
Time:		20:44:47			Log-Likelihood:	-139.70
No. Observations:		100			AIC:	283.4
Df Residuals:		98			BIC:	288.6
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
const	5.1699	0.225	22.994	0.000	4.724	5.616
x1	0.9731	0.040	24.161	0.000	0.893	1.053
Omnibus:		0.322	Durbin-Watson:		1.851	
Prob(Omnibus):		0.851	Jarque-Bera (JB):		0.099	
Skew:		-0.067	Prob(JB):		0.951	
Kurtosis:		3.078	Cond. No.		13.0	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [15]:

```
'''
코드는 선형 회귀 모델을 사용하여 데이터를
학습하고 평가하는 과정을 수행합니다.
```

1. `reg_lr = LinearRegression()`:
먼저, 선형 회귀 모델 객체를 생성합니다.
이 모델은 주어진 독립 변수에 대해
종속 변수의 선형 관계를 학습합니다.
2. `X_train, X_test, y_train, y_test`
= `train_test_split(np.expand_dims(X, axis=-1),`
 `y, train_size=0.8, random_state=123)`:
`train_test_split` 함수를 사용하여 데이터를
학습용(`train`)과 테스트용(`test`)으로 분할합니다.
이 함수는 데이터를 주어진 비율
(여기서는 80% 학습, 20% 테스트)로 나누며,
`random_state` 매개변수를 사용하여
분할 과정에서의 무작위성을 제어합니다.
`np.expand_dims(X, axis=-1)`는 입력 데이터 `X`를
한 차원 늘려줍니다.
이는 `LinearRegression` 모델이 입력 데이터로
2D 배열을 기대하기 때문에 수행됩니다.
3. `reg_lr.fit(X_train, y_train)`:
선형 회귀 모델을 학습 데이터에 맞춥니다.
이는 모델이 주어진 독립 변수와 종속 변수 간의
관계를 학습하는 과정입니다.
4. `mean_squared_error(y_test, reg_lr.predict(X_test))`:
`mean_squared_error` 함수를 사용하여
테스트 데이터에서의 성능을 평가합니다.
이 함수는 실제 값(`y_test`)과
모델의 예측값(`reg_lr.predict(X_test)`) 간의
평균 제곱 오차(Mean Squared Error)를 계산합니다.
MSE는 모델의 예측이 실제 값과 얼마나 다른지를
나타내는 지표로, 값이 작을수록 모델의 예측이 정확합니다.

따라서 코드는 선형 회귀 모델을 사용하여
데이터를 학습하고, 테스트 데이터에서의
평균 제곱 오차를 계산하여 모델의 성능을 평가합니다.
'''

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

reg_lr = LinearRegression()
```

```
X_train, X_test, y_train, y_test \
    = train_test_split(np.expand_dims(X, axis=-1),
                        y, train_size=0.8,
                        random_state=123)
reg_lr.fit(X_train, y_train)
mean_squared_error(y_test, reg_lr.predict(X_test))
```

Out[15]: 1.2583850943570876

In [16]: Image('154.png')

Out[16]: **0. 데이터셋 소개**

Abalone

[Abalone](#) 데이터셋: 전복의 생육 정보 데이터입니다.

Name	Data Type	Meas.	Description
Sex	nominal		M, F, and I (infant)
Length	continuous	mm	Longest shell measurement
Diameter	continuous	mm	perpendicular to length
Height	continuous	mm	with meat in shell
Whole	continuous	grams	whole abalone
Shucked	continuous	grams	weight of meat
Viscera	continuous	grams	gut weight (after bleeding)
Shell	continuous	grams	after being dried
Rings	integer		+1.5 gives the age in years

분류와 회귀 문제 모두를 다루어 볼 수 있는 데이터셋입니다.

```
In [17]: df_abalone = pd.read_csv('data/abalone.csv')
df_abalone.head()
```

```
Out[17]:
```

	Sex	Length	Diameter	Height	Whole	Shucked	Viscera	Shell	Rings
0	M	0.455	0.365	0.095	0.5140	0.2245	0.1010	0.150	15
1	M	0.350	0.265	0.090	0.2255	0.0995	0.0485	0.070	7
2	F	0.530	0.420	0.135	0.6770	0.2565	0.1415	0.210	9
3	M	0.440	0.365	0.125	0.5160	0.2155	0.1140	0.155	10
4	I	0.330	0.255	0.080	0.2050	0.0895	0.0395	0.055	7

```
In [18]: df_berry = pd.read_csv('data/wild_blueberry.csv', index_col='Row#')
df_berry.head()
```

```
Out[18]:
```

	clonesize	honeybee	bumbles	andrena	osmia	MaxOfUpperTRange	MinOfUpperTRange	AverageOfUpperTRange	MaxOfLowerTF
Row#									
0	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
1	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
2	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
3	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
4	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	



```
In [19]: Image('155.png')
```

Out[19]:

1. 선형 회귀 모델

학습 방법

1. 일반 최소 제곱법(Ordinary Least Square)

정규 방정식을 이용하여 계수를 구합니다.

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

$X^T X$ 그람 행렬의 역행렬을 구해야 하므로 변수의 개수가 많아질 수록 학습 시간이 오래 걸립니다.

sklearn.linear_model.LinearRegression

주요 하이퍼 파라미터

하이퍼 파라미터	설명
fit_intercept	절편 사용여부

In [20]: `Image('156.png')`

Out[20]: 2. 경사하강법(Gradient Descent)

sklearn.linear_model.SGDRegressor

- 주요 하이퍼 파라미터

하이퍼 파라미터	설명
max_iter	경사하강법에서 데이터 학습 반복수, Epoch
shuffle	학습시 데이터셋 섞음 여부
learning_rate	학습율, 'constant', 'optimal', 'invscaling', 'adaptive'
alpha	규제 계수
eta0	초기 학습율
early_stopping	조기 종료 여부
validation_fraction	조기 종료를 판단하기 위한 성능 측정셋의 비율
n_iter_no_change	조기 종료를 판단시 성능 변화가 없는 반복수

```
In [21]: '''
`SGDRegressor`는 사이킷런의 `linear_model` 모듈에 포함된
확률적 경사 하강법(Stochastic Gradient Descent, SGD)을
사용한 선형 회귀 모델입니다.
이 모델은 대규모 데이터셋에서도 효율적으로
학습할 수 있도록 설계되었습니다.

### `SGDRegressor`의 주요 인자
1. `loss`:
  - 기본값: `'squared_loss'`
  - 설명: 손실 함수를 지정합니다.
    - `'squared_loss'`: 제곱 오차 (기본값)
    - `'huber'`: 허버 손실 함수
    - `'epsilon_insensitive'`: 절대값 손실 함수
    - `'squared_epsilon_insensitive'`:
```

제공 절대값 손실 함수

2. ``penalty``:
 - 기본값: ``'l2'``
 - 설명: 사용될 페널티(정규화) 유형을 지정합니다.
사용할 수 있는 값으로는 ``'none'``, ``'l2'``,
``'l1'``, ``'elasticnet'`` 등이 있습니다.
 - ``'l2'``: L2 정규화 (기본값)
 - ``'l1'``: L1 정규화
 - ``'elasticnet'``: L1과 L2의 조합
3. ``alpha``:
 - 기본값: ``0.0001``
 - 설명: 정규화 항의 계수입니다.
정규화 강도를 조절합니다.
4. ``l1_ratio``:
 - 기본값: ``0.15``
 - 설명: 엘라스틱넷(ElasticNet) 정규화 사용 시
L1과 L2의 가중치 비율입니다.
5. ``fit_intercept``:
 - 기본값: ``True``
 - 설명: 모델에 상수항(절편)을 포함할지
여부를 지정합니다.
6. ``max_iter``:
 - 기본값: ``1000``
 - 설명: 학습 시도의 최대 반복 횟수입니다.
7. ``tol``:
 - 기본값: ``1e-3``
 - 설명: 수렴을 판단할 허용 오차입니다.
8. ``shuffle``:
 - 기본값: ``True``
 - 설명: 각 에포크(epoch)마다 데이터를
섞을지 여부를 지정합니다.
9. ``verbose``:
 - 기본값: ``0``
 - 설명: 학습 과정 중 출력 메시지의
상세 수준입니다.
10. ``epsilon``:
 - 기본값: ``0.1``
 - 설명: 허버 손실과 절대값 손실 함수에
사용할 감마 값을 지정합니다.
11. ``learning_rate``:
 - 기본값: ``'invscaling'``

```

- 설명: 학습률 스케줄을 지정합니다.
  사용할 수 있는 값으로는
  ``constant``, ``optimal``, ``invscaling``,
  ``adaptive`` 등이 있습니다.
12. `eta0`:
  - 기본값: `0.01`
  - 설명: 초기 학습률입니다.
13. `power_t`:
  - 기본값: `0.25`
  - 설명: 학습률 스케줄이 ``invscaling``일
    때의 지수입니다.

### 리턴값
`SGDRegressor` 객체는 학습된 모델을 반환합니다.
이 객체는 여러 가지 메서드와 속성을 제공하여
모델의 학습 및 평가를 지원합니다.

### 주요 메서드

- `fit(X, y)`:
  모델을 학습 데이터 `X`와 레이블 `y`에
  맞추어 학습합니다.
- `predict(X)`:
  학습된 모델을 사용하여 입력 데이터 `X`에
  대한 예측값을 반환합니다.
- `partial_fit(X, y)`:
  부분적으로 데이터를 사용하여 모델을 학습합니다.
  매우 큰 데이터셋에 대해 점진적으로
  학습할 때 유용합니다.
- `score(X, y)`:
  입력 데이터 `X`와 실제 값 `y`에 대한
  결정 계수  $R^2$  점수를 반환합니다.

### 주요 속성

- `coef_`: 학습된 모델의 계수를 나타냅니다.
- `intercept_`: 학습된 모델의 절편을 나타냅니다.
- `n_iter_`: 학습을 위해 수행된 에포크 수를 나타냅니다.
...
...

```

코드는 `SGDRegressor`를 사용하여 선형 회귀 모델을 학습하고, 테스트 데이터를 통해 모델의 성능을 평가합니다.

1. `reg_sgd_lr = SGDRegressor(`:`
 - `SGDRegressor` 객체를 생성합니다.
이 객체는 확률적 경사 하강법 (Stochastic Gradient Descent, SGD)을 사용하여 선형 회귀 모델을 학습합니다.
 2. `alpha=0,`:`
 - 규제 계수를 0으로 설정합니다. 이는 정규화(페널티)를 사용하지 않는다는 의미로, `LinearRegression`의 기본 설정과 동일한 조건으로 학습되도록 합니다.
 3. `shuffle=True,`:`
 - 각 에포크(epoch)마다 데이터를 무작위로 섞습니다.
이는 데이터가 편중된 경우에도 모델이 더 고르게 학습되도록 도와줍니다.
 4. `early_stopping=False`:`
 - 조기 종료(Early Stopping)를 사용하지 않도록 설정합니다. 조기 종료는 학습 도중 검증 손실이 더 이상 감소하지 않을 때 학습을 멈추는 기술이지만, 여기서는 이를 사용하지 않습니다.
- 학습 데이터를 사용하여 `SGDRegressor` 모델을 학습시킵니다. `fit` 메서드는 주어진 데이터 (`X_train`, `y_train`)에 맞추어 모델의 파라미터를 최적화합니다.
- 테스트 데이터를 사용하여 모델의 성능을 평가합니다.
- `reg_sgd_lr.predict(X_test)`:`
학습된 모델을 사용하여 테스트 데이터(`X_test`)에 대한 예측값을 생성합니다.
 - `mean_squared_error(y_test, reg_sgd_lr.predict(X_test))`:`
예측값과 실제 값(`y_test`) 간의 평균 제곱 오차 (Mean Squared Error, MSE)를 계산합니다.
MSE는 예측값이 실제 값과 얼마나 차이가 나는지를 나타내는 지표로, 값이 작을수록 모델의 예측이 정확함을 의미합니다.

요약

이 코드는 다음과 같은 과정을 수행합니다:

1. `SGDRegressor` 객체를 생성하고, 규제 계수를 0으로 설정하여 정규화를 사용하지 않도록 합니다.
2. 각 에포크마다 데이터를 무작위로 섞어 학습 데이터의 편종을 방지합니다.
3. 학습 데이터를 사용하여 모델을 학습시킵니다.
4. 테스트 데이터를 사용하여 모델의 예측 성능을 평가하고, 평균 제곱 오차를 계산합니다.

이를 통해 모델이 학습 데이터에 대한 패턴을 얼마나 잘 학습했는지, 그리고 학습된 모델이 테스트 데이터에 대해 얼마나 정확한 예측을 수행하는지를 평가할 수 있습니다.

```
from sklearn.linear_model import SGDRegressor

reg_sgd_lr = SGDRegressor(
    # LinearRegression의 규제 계수는 0 입니다.
    # 동일한 조건으로 학습되도록, 규제 계수를 0으로 설정합니다.
    alpha=0,
    random_state=123,
    # 데이터가 편중되었을 경우를 대비하여
    # 고루 섞이도록 설정합니다.
    shuffle=True,
    # 조기 종료 루틴은 사용하지 않습니다.
    early_stopping=False
)

reg_sgd_lr.fit(X_train, y_train)

mean_squared_error(y_test, reg_sgd_lr.predict(X_test))
```

Out[21]: 1.1312637557062695

- 경사하강법을 통한 선형회귀모델의

안정적인 학습을 위해서는 입력 변수의

표준화 또는 정규화가 필요합니다.

[Ex.2]

df_abalone으로 연령(Rings)를 대상변수로 하고

나머지 변수를 입력 변수로 하여

경사하강법으로 선형회귀 모델을 학습합니다.

연속형 변수의 표준화 여부가 모델 학습에

어떤 영향을 미치는지 확인해봅니다.

```
In [22]: '''
코드는 `df_abalone` 데이터프레임에서 종속 변수를
제외한 독립 변수들의 기술 통계량을 출력합니다.
이를 통해 각 독립 변수의 스케일과 분포를 확인할 수 있습니다.

1. `X_cols = [i for i in df_abalone.columns
               if i not in ['Rings']]`:
- `df_abalone` 데이터프레임의 열 이름 중에서
  `Rings` 열을 제외한 열 이름을 리스트로 만듭니다.
- 리스트 컴프리헨션을 사용하여 이 작업을 수행합니다.
  - `df_abalone.columns`는 데이터프레임의
    모든 열 이름을 반환합니다.
  - `if i not in ['Rings']` 조건을 사용하여
    `Rings` 열을 제외합니다.

예를 들어, `df_abalone` 데이터프레임의 열 이름이
`['Length', 'Diameter', 'Height', 'Whole_weight',
 'Shucked_weight', 'Viscera_weight', 'Shell_weight',
 'Rings']`라면,
`X_cols`는 `['Length', 'Diameter', 'Height', 'Whole_weight',
 'Shucked_weight', 'Viscera_weight', 'Shell_weight']`가 됩니다.
'''

# 입력 변수의 스케일 확인하기 위해 기술 통계량을 출력해봅니다.
X_cols = [i for i in df_abalone.columns \
```

```
if i not in ['Rings']]
df_abalone[X_cols].describe()
```

Out[22]:

	Length	Diameter	Height	Whole	Shucked	Viscera	Shell
count	4177.000000	4177.000000	4177.000000	4177.000000	4177.000000	4177.000000	4177.000000
mean	0.523992	0.407881	0.139516	0.828742	0.359367	0.180594	0.238831
std	0.120093	0.099240	0.041827	0.490389	0.221963	0.109614	0.139203
min	0.075000	0.055000	0.000000	0.002000	0.001000	0.000500	0.001500
25%	0.450000	0.350000	0.115000	0.441500	0.186000	0.093500	0.130000
50%	0.545000	0.425000	0.140000	0.799500	0.336000	0.171000	0.234000
75%	0.615000	0.480000	0.165000	1.153000	0.502000	0.253000	0.329000
max	0.815000	0.650000	1.130000	2.825500	1.488000	0.760000	1.005000

In [26]:

```
'''
코드는 `SGDRegressor`를 사용하여 `df_abalone` 데이터프레임의
데이터를 학습하고, 모델의 예측 성능을 평가합니다.

1. 독립 변수와 연속형 변수 설정:
- `X_cols`: `df_abalone` 데이터프레임의 모든 열 이름 중
  `Rings`를 제외한 열 이름을 포함하는 리스트를 만듭니다.
- `cont_val`: `df_abalone` 데이터프레임의 모든 열 이름 중
  `Sex`와 `Rings`를 제외한 열 이름을 포함하는 리스트를
  만듭니다. 이 리스트는 연속형 변수를 나타냅니다.

2. 컬럼 변환기 설정:
- `ColumnTransformer`:
  여러 열에 대해 다른 전처리 방법을 적용할 수 있도록 합니다.
- `('std', 'passthrough', cont_val)`:
  `cont_val` 리스트에 포함된 연속형 변수는
  별도의 전처리 없이 그대로 전달됩니다(`'passthrough'`).
- `('ohe', OneHotEncoder(drop='first'), ['Sex'])`:
  `Sex` 열은 원-핫 인코딩(One-Hot Encoding)을 사용하여
  가변수(dummy variable)로 변환되며,
```

첫 번째 범주는 제외됩니다(``drop='first'``).

이는 다중 공선성 문제를 방지하기 위함입니다.

3. 파이프라인 설정 및 모델 학습:

- ``make_pipeline(ct, SGDRegressor(max_iter=100))``:
컬럼 변환기(``ct``)와 ``SGDRegressor``를
파이프라인으로 결합합니다.
파이프라인을 사용하면 데이터 전처리와 모델 학습을
연속적으로 수행할 수 있습니다.
- ``SGDRegressor(max_iter=100)``:
``SGDRegressor`` 모델을 설정하며, 최대 반복 횟수
(``max_iter``)는 100으로 설정합니다.
- ``reg_sgd_lr.fit(df_abalone[X_cols], df_abalone['Rings'])``:
``df_abalone`` 데이터프레임의 독립 변수(``X_cols``)와
종속 변수(``Rings``)를 사용하여 모델을 학습시킵니다.

4. 모델 성능 평가:

- ``mean_squared_error(df_abalone['Rings'],
reg_sgd_lr.predict(df_abalone[X_cols]))``:
학습된 모델을 사용하여 ``df_abalone`` 데이터프레임의
독립 변수(``X_cols``)에 대한 예측값을 생성하고,
실제 값(``Rings``)과 비교하여 평균 제곱 오차
(Mean Squared Error, MSE)를 계산합니다.
MSE는 모델의 예측 성능을 평가하는 지표로,
값이 작을수록 예측이 실제 값과 가까움을 나타냅니다.

요약

이 코드는 다음과 같은 작업을 수행합니다:

1. 독립 변수와 연속형 변수를 설정합니다.
2. 컬럼 변환기를 설정하여 연속형 변수는 그대로 통과시키고,
범주형 변수(``Sex``)는 원-핫 인코딩합니다.
3. 파이프라인을 사용하여 데이터 전처리와 모델 학습을 결합합니다.
4. 모델을 학습시키고, 학습된 모델을 사용하여
평균 제곱 오차를 계산하여 모델의 예측 성능을 평가합니다.

...

...

``ColumnTransformer``는 사이킷런(sklearn)의 데이터 전처리
모듈에서 제공하는 유용한 도구로, 데이터프레임의 여러 열에
대해 서로 다른 전처리 방법을 적용할 수 있게 해줍니다.
이는 특히 다양한 타입의 데이터가 섞여 있는 경우 유용합니다.

주요 인자

`ColumnTransformer`의 주요 인자는 다음과 같습니다:

1. `transformers`:

- 타입: 리스트
- 설명: 각 열 또는 열 그룹에 적용할 변환기와 해당 열의 이름을 포함하는 리스트입니다. 각 요소는 다음 세 부분으로 구성됩니다:
 - 이름: 변환기의 이름 (문자열)
 - 변환기: 변환기 객체, 'drop' 또는 'passthrough' 문자열
 - 열 이름: 적용할 열의 이름(리스트나 배열 형태)

예시:

```
transformers=[
    ('num', StandardScaler(), ['age', 'income']),
    ('cat', OneHotEncoder(), ['gender', 'occupation'])
]
```

2. `remainder` (선택적):

- 타입: 문자열, 'drop' 또는 'passthrough'
- 기본값: 'drop'
- 설명: 지정되지 않은 열에 대해 처리 방법을 지정합니다. 'drop'은 해당 열을 제거하고, 'passthrough'는 해당 열을 원본 상태로 유지합니다.

예시:

```
remainder='passthrough'
```

3. `sparse\_threshold` (선택적):

- 타입: float, 기본값: 0.3
- 설명: 희소 행렬(sparse matrix)의 밀도 임계값입니다. 변환 결과가 이 임계값보다 밀도가 낮다면 희소 행렬로 반환됩니다.

4. `n\_jobs` (선택적):

- 타입: int, 기본값: None
- 설명: 병렬 처리를 위한 CPU 코어 수입니다. -1로 설정하면 모든 가용 코어를 사용합니다.

5. `transformer\_weights` (선택적):

- 타입: 딕셔너리, 기본값: None
- 설명: 변환기에 가중치를 부여하여 가중합을 계산할 때 사용합니다.

6. `verbose` (선택적):

- 타입: bool, 기본값: False
- 설명: 변환기에서 작업의 진행 상황을

출력할지 여부를 지정합니다.

```

### 리턴값
`ColumnTransformer` 객체는 변환된 데이터를
반환합니다. 주로 다음과 같은 메서드를 사용합니다:
- `fit(X, y=None)` :
    데이터에 대해 변환기를 학습시킵니다.
- `fit_transform(X, y=None)` :
    데이터에 대해 변환기를 학습시키고 변환을 적용합니다.
- `transform(X)` :
    학습된 변환기를 사용하여 데이터를 변환합니다.
- `inverse_transform(X)` :
    변환된 데이터를 원래 형태로 되돌립니다
    (일부 변환기에만 적용 가능).
...

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, \
    OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_squared_error

X_cols \
    = [i for i in df_abalone.columns \
        if i not in ['Rings']]
cont_val \
    = [i for i in df_abalone.columns \
        if i not in ['Sex', 'Rings']]

ct = ColumnTransformer([
    ('std', 'passthrough', cont_val),
    # 성별은 범주형 변수이고 이를 첫번째 범주를
    # 제외하여 가변수화합니다.
    ('ohe', OneHotEncoder(drop='first'), ['Sex'])
])

reg_sgd_lr = make_pipeline(ct, SGDRegressor(max_iter=100))
reg_sgd_lr.fit(df_abalone[X_cols], df_abalone['Rings'])
mean_squared_error(df_abalone['Rings'], \
    reg_sgd_lr.predict(df_abalone[X_cols]))

```

Out[26]: 5.309952272317429

```
In [10]: '''
코드는 `ColumnTransformer`와 `SGDRegressor`를 사용하여
`df_abalone` 데이터프레임의 데이터를 전처리하고
학습시키며, 모델의 예측 성능을 평가합니다.

### 1. 컬럼 변환기 설정
- `ColumnTransformer` 객체 `ct`를 생성합니다.
  - ``std``: `cont_val` 리스트에 포함된 연속형 변수들에
    `StandardScaler`를 적용합니다. `StandardScaler`는
    각 변수의 평균을 0, 표준 편차를 1로 표준화합니다.
  - ``ohe``: `Sex` 열에 `OneHotEncoder(drop='first')`를 적용합니다.
    `OneHotEncoder`는 범주형 변수를 원-핫 인코딩하여
    가변수(dummy variable)로 변환하며,
    첫 번째 범주를 제외합니다(`drop='first'`).
    이는 다중 공선성 문제를 방지하기 위함입니다.

### 2. 파이프라인 설정 및 모델 학습
- `make_pipeline(ct, SGDRegressor(max_iter=100))`:
  - `ct` (컬럼 변환기)와 `SGDRegressor`를
    파이프라인으로 결합합니다. 파이프라인을 사용하면 데이터
    전처리와 모델 학습을 연속적으로 수행할 수 있습니다.
  - `SGDRegressor(max_iter=100)`:
    `SGDRegressor` 모델을 설정하며, 최대 반복 횟수(`max_iter`)는
    100으로 설정합니다. `SGDRegressor`는 확률적 경사 하강법
    (Stochastic Gradient Descent)을 사용하여 모델을 학습합니다.
- `reg_sgd_lr.fit(df_abalone[X_cols], df_abalone['Rings']):`
  - `df_abalone` 데이터프레임의 독립 변수(`X_cols`)와
    종속 변수(`Rings`)를 사용하여 모델을 학습시킵니다.
    이 과정에서 파이프라인의 첫 번째 단계(`ColumnTransformer`)가
    먼저 적용되어 데이터가 전처리되고, 전처리된 데이터가
    `SGDRegressor`에 의해 학습됩니다.

### 3. 모델 성능 평가
- `reg_sgd_lr.predict(df_abalone[X_cols]):`
  - 학습된 모델을 사용하여 `df_abalone` 데이터프레임의
    독립 변수(`X_cols`)에 대한 예측값을 생성합니다.
- `mean_squared_error(df_abalone['Rings'],
  reg_sgd_lr.predict(df_abalone[X_cols]))`:
```

- 예측값과 실제 값(`Rings`) 간의 평균 제곱 오차 (Mean Squared Error, MSE)를 계산합니다. MSE는 모델의 예측 성능을 평가하는 지표로, 값이 작을수록 모델의 예측이 실제 값에 가까움을 나타냅니다.

요약

1. `ColumnTransformer`를 사용하여 연속형 변수는 표준화하고, 범주형 변수(`Sex`)는 원-핫 인코딩합니다.
2. 파이프라인을 사용하여 데이터 전처리와 모델 학습을 결합합니다.
3. `SGDRegressor`를 사용하여 모델을 학습시킵니다.
4. 학습된 모델을 사용하여 예측을 수행하고, 평균 제곱 오차를 계산하여 모델의 예측 성능을 평가합니다.

이를 통해 데이터 전처리와 모델 학습을 효율적으로 수행하고, 모델의 성능을 평가할 수 있습니다.

```
ct = ColumnTransformer([
    # 연속형 변수들을 표준화합니다.
    ('std', StandardScaler(), cont_val),
    ('ohe', OneHotEncoder(drop='first'), ['Sex'])
])

reg_sgd_lr = make_pipeline(ct, SGDRegressor(max_iter=100))
reg_sgd_lr.fit(df_abalone[X_cols], \
               df_abalone['Rings'])
mean_squared_error(df_abalone['Rings'], \
                   reg_sgd_lr.predict(df_abalone[X_cols]))
```

Out[10]: 4.838092242471089

두 경우 모두 100번의 반복수를 주고 동일하게 설정했지만,
표준화를 하지 않았을 경우 100번 안에 수렴하지 못했다는 점에서,
효율적으로 동작하지 못함을 확인할 수 있습니다.

In [29]: `Image('157.png')`

Out[29]: **규제(Regularization)**

회귀 계수의 절대 크기를 억제하는 방법입니다. Loss 함수에 규제를 더해줍니다.

1. L1 규제: 계수(Coefficient)에 절대값의 합. (α : 규제 계수)

$$\text{L1 규제(L1 Regularization)} = \alpha \sum_{i=1}^n \|\beta_i\|_1 = \alpha \sum_{i=1}^n |\beta_i|$$

2. L2 규제: 계수(Coefficient)에 제곱의 합. (α : 규제 계수)

$$\text{L2 규제(L2 Regularization)} = \alpha \sum_{i=1}^n \|\beta_i\|_2^2 = \alpha \sum_{i=1}^n \beta_i^2$$

손실(Loss) 함수에 규제를 포함해서 비용 함수(Cost Function)라고 합니다.

규제를 입력 변수에 해당하는 계수별로 균일하게

적용시키기 위해서는 표준화 또는 정규화가 필요합니다.

이를 수행하지 않는다면, 스케일이 큰 변수는 규제를

상대적으로 적게 받게 되고 스케일이 작은 변수는 상대적으로 크게 받게 됩니다.

```
In [11]: '''
코드는 두 함수  $\|x\|$ 와  $x^2$ 의 그래프를
그리기 위한 것입니다.

### 1. 함수 그래프 그리기
- `plt.plot(x, np.abs(x))`는  $\|x\|$  함수의 그래프를 그립니다.
  - `np.abs(x)`는 `x` 배열의 각 원소에 대해
    절대값을 계산합니다.
  - 이 계산된 값을 y축으로 사용하여 그래프를 그립니다.
- `plt.plot(x, np.square(x))`는  $x^2$  함수의 그래프를 그립니다.
  - `np.square(x)`는 `x` 배열의 각 원소에 대해
    제곱값을 계산합니다.
```

- 이 계산된 값을 y 축으로 사용하여 그래프를 그립니다.

2. 세로선 추가

- `plt.axvline(0, linestyle='dotted')`는 $x=0$ 위치에 세로선을 추가합니다.
- `axvline` 함수는 수직선을 그릴 때 사용됩니다.
- `0`은 선이 그려질 x 좌표를 나타냅니다.
- `linestyle='dotted'`는 선을 점선으로 설정합니다.

전체 코드의 동작

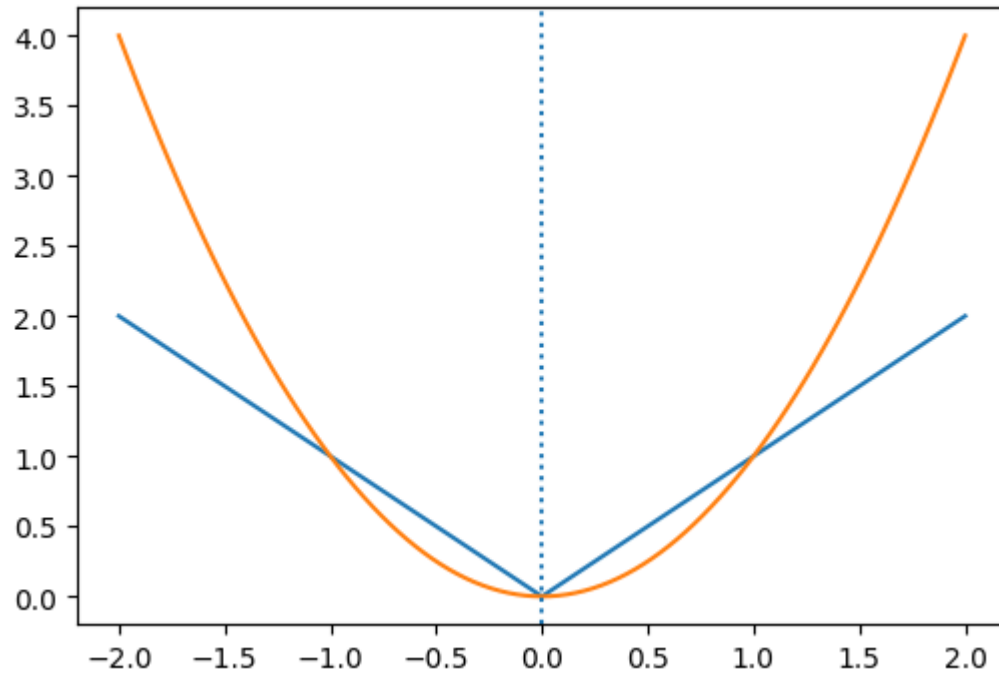
1. 그림 크기를 6인치 x 4인치로 설정합니다.
2. -2에서 2까지 101개의 균등 간격의 x 값을 생성합니다.
3. $\backslash(|x|\backslash)$ 함수와 $\backslash(x^2\backslash)$ 함수의 그래프를 각각 그립니다.
4. $x=0$ 위치에 점선을 그려 y 축을 강조합니다.

결과적으로, $\backslash(|x|\backslash)$ 함수는 두 직선 형태의 그래프를,
 $\backslash(x^2\backslash)$ 함수는 포물선 형태의 그래프를 가지며,
 두 그래프가 함께 표시됩니다.

중간의 점선은 $x=0$ 위치를 나타내어
 두 함수의 대칭성을 강조합니다.

...

```
plt.figure(figsize=(6, 4))
x = np.linspace(-2, 2, 101)
plt.plot(x, np.abs(x))
plt.plot(x, np.square(x))
plt.axvline(0, linestyle='dotted')
plt.show()
```



L2 규제는 크기(Magnitude)가 1보다 크면 L1 보다 규제의 강도가 세지는 반면, 1보다 작으면 규제의 강도가 L1에 비해 작아 집니다.

L2 규제는 계수가 L1 규제에 비해 작아 질수록 급격하게 규제가 줄어 듭니다.

L1 규제는 계수가 0이 되는 경우가 L2에 비해 빈번합니다.

```
In [33]: '''
코드는 `df_abalone` 데이터프레임을 전처리하고
학습용 데이터와 테스트 데이터를 나누기 위한 것입니다.

### 1. 독립 변수와 연속형 변수 설정
- `X_cols`:
  `df_abalone` 데이터프레임의 모든 열 이름 중
  'Rings'를 제외한 열 이름을 리스트로 만듭니다.
```

```

    이는 모델의 독립 변수(입력 변수)로 사용됩니다.
- `cont_val`:
    `df_abalone` 데이터프레임의 모든 열 이름 중
    'Sex'와 'Rings'를 제외한 열 이름을 리스트로 만듭니다.
    이는 연속형 변수들입니다.

### 2. 컬럼 변환기 설정
- `ColumnTransformer` 객체 `ct`를 생성합니다.
- `('std', StandardScaler(), cont_val)`:
    `cont_val` 리스트에 포함된 연속형 변수들에 대해
    `StandardScaler`를 적용합니다. `StandardScaler`는
    각 변수의 평균을 0, 표준 편차를 1로 표준화합니다.
- `('ohe', OneHotEncoder(drop='first'), ['Sex'])`:
    'Sex' 열에 대해 `OneHotEncoder(drop='first')`를 적용합니다.
    `OneHotEncoder`는 범주형 변수를 원-핫 인코딩하여
    가변수(dummy variable)로 변환하며,
    첫 번째 범주를 제외합니다(`drop='first'`).
    이는 다중 공선성 문제를 방지하기 위함입니다.

### 3. 변환된 변수 이름 설정
- `X_ct_name`:
    전처리된 데이터프레임의 새로운 열 이름을 리스트로 만듭니다.
- `[i + '_std' for i in cont_val]`:
    `cont_val` 리스트에 있는 각 열 이름에
    '_std'를 추가하여 표준화된 변수 이름을 만듭니다.
- `['Sex_I', 'Sex_M']`:
    원-핫 인코딩된 'Sex' 변수의 새로운 열 이름입니다.
    'Sex_I'는 'Sex' 변수에서 'I' (Infant)를 나타내고,
    'Sex_M'는 'M' (Male)을 나타냅니다.
    'F' (Female)는 원-핫 인코딩에서
    제외된 첫 번째 범주입니다.

### 4. 학습용 데이터와 테스트 데이터 분할
- `train_test_split(df_abalone, train_size=0.8,
    random_state=123)`:
    `df_abalone` 데이터프레임을 학습용 데이터와
    테스트 데이터로 분할합니다.
- `train_size=0.8`:
    데이터의 80%를 학습용 데이터로 사용하고,
    나머지 20%를 테스트 데이터로 사용합니다.

```


전체 코드의 동작 요약

1. ``X_cols``와 ``cont_val`` 리스트를 생성하여
모델의 독립 변수와 연속형 변수를 설정합니다.
2. ``ColumnTransformer``를 설정하여 연속형 변수는
표준화하고, 범주형 변수(``Sex``)는 원-핫 인코딩합니다.
3. 변환된 데이터프레임의 새로운 열 이름을 설정합니다.
4. ``df_abalone`` 데이터프레임을 학습용 데이터
(``df_train``)와 테스트 데이터(``df_test``)로 분할합니다.

이 과정을 통해 데이터 전처리와 학습용 데이터 및
테스트 데이터의 분할을 완료하여,
이후의 모델 학습과 평가를 위한 준비를 마칩니다.
'''

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split

X_cols = [i for i in df_abalone.columns \
           if i not in ['Rings']]
cont_val = [i for i in df_abalone.columns \
            if i not in ['Sex', 'Rings']]

ct = ColumnTransformer([
    ('std', StandardScaler(), cont_val),
    ('ohe', OneHotEncoder(drop='first'), ['Sex'])
])

X_ct_name = [i + '_std' for i in cont_val] \
             + ['Sex_I', 'Sex_M']

df_train, df_test = train_test_split(df_abalone,
                                     train_size=0.8,
                                     random_state=123)
```

In [13]: `df_train.head()`

Out[13]:

	Sex	Length	Diameter	Height	Whole	Shucked	Viscera	Shell	Rings
2746	I	0.485	0.365	0.140	0.4475	0.1895	0.0925	0.2305	8
853	M	0.570	0.455	0.150	0.9520	0.3895	0.2155	0.2745	9
1809	F	0.640	0.475	0.140	1.0725	0.4895	0.2295	0.3100	8
3419	M	0.590	0.470	0.135	1.1685	0.5390	0.2790	0.2800	8
1199	F	0.720	0.580	0.195	2.1030	1.0265	0.4800	0.5375	10

In [34]:

```
Image('158.png')
```

Out[34]: **Ridge**

- L2 규제를 손실함수에 더한 선형 회귀 모델 입니다.

sklearn.linear_model.Ridge

$$L_{ridge}(y, \hat{y}) = \frac{1}{2n} ||y - \hat{y}||_2^2 + \alpha ||\hat{\beta}||_2^2$$

- 주요 하이퍼 파라미터

하이퍼 파라미터	설명
alpha	규제 계수

Ridge 회귀는 정규방정식으로도 도출할 수 있습니다.

$$S(\beta) = \sum_{i=1}^n (y_i - X_i \beta)^2 + \alpha \sum_{i=1}^k \beta_i^2 = ||y - X\beta||^2 + \alpha \beta^T \beta$$
$$||y - X\beta||^2 + \alpha \beta^T \beta = (y - X\beta)^T (y - X\beta) + \alpha \beta^T \beta$$
$$\frac{dS(\hat{\beta})}{d\hat{\beta}} = -X^T y + X^T X \hat{\beta} + \alpha \hat{\beta}$$
$$-X^T y + X^T X \hat{\beta} + \alpha \hat{\beta} = 0$$
$$\hat{\beta} = (X^T X + \alpha I)^{-1} X^T y, I: \text{크기가 입력 변수의 수인 단위 행렬}$$

[Ex.3]

df_abalone에서 성별(Sex)을 사전 순으로

첫 번째 범주인 'F'를 제외하고 가변수화 합니다.

80%는 학습 데이터 df_train으로, 20%는 평가 데이터 df_test로 나눕니다.

Ridge 모델의 규제 계수(alpha)에 따라 df_train과 df_test에 대한 평균제곱오차 (Mean Squared Error)를 비교해 봅니다.

alphas = [1e-2, 1e-1, 1, 1e1, 1e2, 1e3]

```
In [14]: '''
코드는 `Ridge` 회귀 모델을 사용하여 다양한 `alpha` 값을
통해 모델을 학습시키고, 학습된 모델의 계수(coefficient)와
학습 및 테스트 데이터에 대한 평균 제곱 오차
(Mean Squared Error, MSE)를 계산하는 과정을 포함합니다.

### 1. 필요한 라이브러리 импорт
- `Ridge`:
    릿지 회귀(Ridge Regression) 모델을 가져옵니다.
    릿지 회귀는 선형 회귀에 L2 규제를 추가하여
    과적합을 방지합니다.
- `mean_squared_error`:
    평균 제곱 오차를 계산하는 함수입니다.

### 2. 알파 값 설정
- `alphas`:
    릿지 회귀의 규제 강도 하이퍼파라미터 `alpha`의
    다양한 값을 설정합니다. 작은 값에서 큰 값까지 다양한
    `alpha`를 설정하여 모델의 성능을 비교합니다.

### 3. 결과를 저장할 리스트 초기화
- `coefs`:
    각 `alpha` 값에 대해 학습된 모델의 계수를
    저장할 리스트입니다.
```

```
- `mse_train`:
    각 `alpha` 값에 대해 학습 데이터의 평균 제곱 오차를
    저장할 리스트입니다.
- `mse_test`:
    각 `alpha` 값에 대해 테스트 데이터의 평균 제곱 오차를
    저장할 리스트입니다.
```

4. 각 알파 값에 대해 모델 학습 및 평가

```
- `for alpha in alphas:`: 각 `alpha` 값에 대해
    루프를 수행합니다.
- `reg_rd = make_pipeline(ct, Ridge(alpha=alpha,
    random_state=123))`:
- `make_pipeline(ct, Ridge(alpha=alpha,
    random_state=123))`:
    컬럼 변환기 `ct`와 릿지 회귀 모델을 파이프라인으로
    결합합니다. `alpha` 값을 현재 루프의 값으로 설정하고,
    `random_state=123`으로 난수 시드를 고정하여
    재현성을 보장합니다.
- `reg_rd.fit(df_train[X_cols], df_train['Rings']):`
- `df_train` 데이터프레임의 독립 변수(`X_cols`)와
    종속 변수(`Rings`)를 사용하여 모델을 학습시킵니다.
    이 과정에서 파이프라인의 첫 번째 단계
    (`ColumnTransformer`)가 먼저 적용되어 데이터가
    전처리된 후 릿지 회귀 모델에 입력됩니다.
- `coefs.append(reg_rd[1].coef_)`:
    - 학습된 모델의 계수를 `coefs` 리스트에 추가합니다.
      `reg_rd[1].coef_`는 파이프라인에서
      두 번째 단계인 `Ridge` 모델의 계수를 의미합니다.
- `mse_train.append(mean_squared_error(df_train['Rings'],
    reg_rd.predict(df_train[X_cols])))`:
    - 학습 데이터에 대한 예측값과 실제값 간의
      평균 제곱 오차를 계산하여 `mse_train`
      리스트에 추가합니다.
- `mse_test.append(mean_squared_error(df_test['Rings'],
    reg_rd.predict(df_test[X_cols])))`:
    - 테스트 데이터에 대한 예측값과 실제값 간의
      평균 제곱 오차를 계산하여
      `mse_test` 리스트에 추가합니다.
```

전체 코드의 동작 요약

1. 다양한 `alpha` 값에 대해

- 릿지 회귀 모델을 설정합니다.
2. 각 ``alpha`` 값에 대해 모델을 학습시키고,
학습된 모델의 계수를 저장합니다.
 3. 각 ``alpha`` 값에 대해 학습 데이터와 테스트 데이터의
평균 제곱 오차를 계산하여 저장합니다.

이 과정을 통해 다양한 규제 강도(``alpha``)에 따른
릿지 회귀 모델의 성능을 비교할 수 있습니다.
이를 통해 최적의 ``alpha`` 값을 선택하고,
모델의 과적합을 방지하는 데 도움을 줄 수 있습니다.

```
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
alphas = [1e-2, 1e-1, 1, 1e1, 1e2, 1e3]
coefs, mse_train, mse_test = list(), list(), list()
for alpha in alphas:
    reg_rd = make_pipeline(ct, Ridge(alpha=alpha,
                                     random_state=123))
    reg_rd.fit(df_train[X_cols], df_train['Rings'])
    coefs.append(reg_rd[1].coef_)
    mse_train.append(\
        mean_squared_error(df_train['Rings'], \
                           reg_rd.predict(df_train[X_cols])))
    mse_test.append(\
        mean_squared_error(df_test['Rings'], \
                           reg_rd.predict(df_test[X_cols])))
```

In [15]: '''
코드는 규제 계수(알파 값)에 따른 회귀 계수를
시각화하는 작업을 수행합니다.

단계별 설명

1. 데이터프레임 생성 및 변환
 - ``pd.DataFrame(coefs, index=pd.Index(alphas, name='alpha'), columns=X_ct_name)``:
 - ``coefs``: 다양한 규제 계수(알파 값)에 대해
계산된 회귀 계수의 배열입니다.
 - ``alphas``: 규제 계수(알파 값)의 리스트입니다.

- ``X_ct_name``: 회귀 모델의 피처(독립 변수) 이름들의 리스트입니다.
- 이 줄은 ``coefs`` 배열을 사용하여 새로운 데이터프레임 ``df_coef``를 생성합니다. 행 인덱스는 ``alphas``이고, 열 이름은 ``X_ct_name``입니다. 인덱스 이름은 ``alpha``로 설정됩니다.
- ``reset_index()``: 인덱스를 열로 변환하여 데이터프레임을 재구성합니다. 즉, ``alpha``가 데이터프레임의 열로 추가됩니다.
- ``melt(id_vars='alpha', value_name='coef', var_name='feature')``:
``melt`` 함수는 데이터를 길게 변환합니다. 여기서:
 - ``id_vars='alpha``:
``alpha`` 열은 식별 변수로 유지됩니다.
 - ``value_name='coef``:
 변환 후 값 열의 이름을 ``coef``로 설정합니다.
 - ``var_name='feature``:
 변환 후 변수 이름을 ``feature``로 설정합니다.

이 변환으로 ``df_coef``는 다음과 같은 형태를 가지게 됩니다:

```
alpha  feature  coef
0.1    feature1  0.5
0.1    feature2  1.2
0.1    feature3 -0.3
...
```

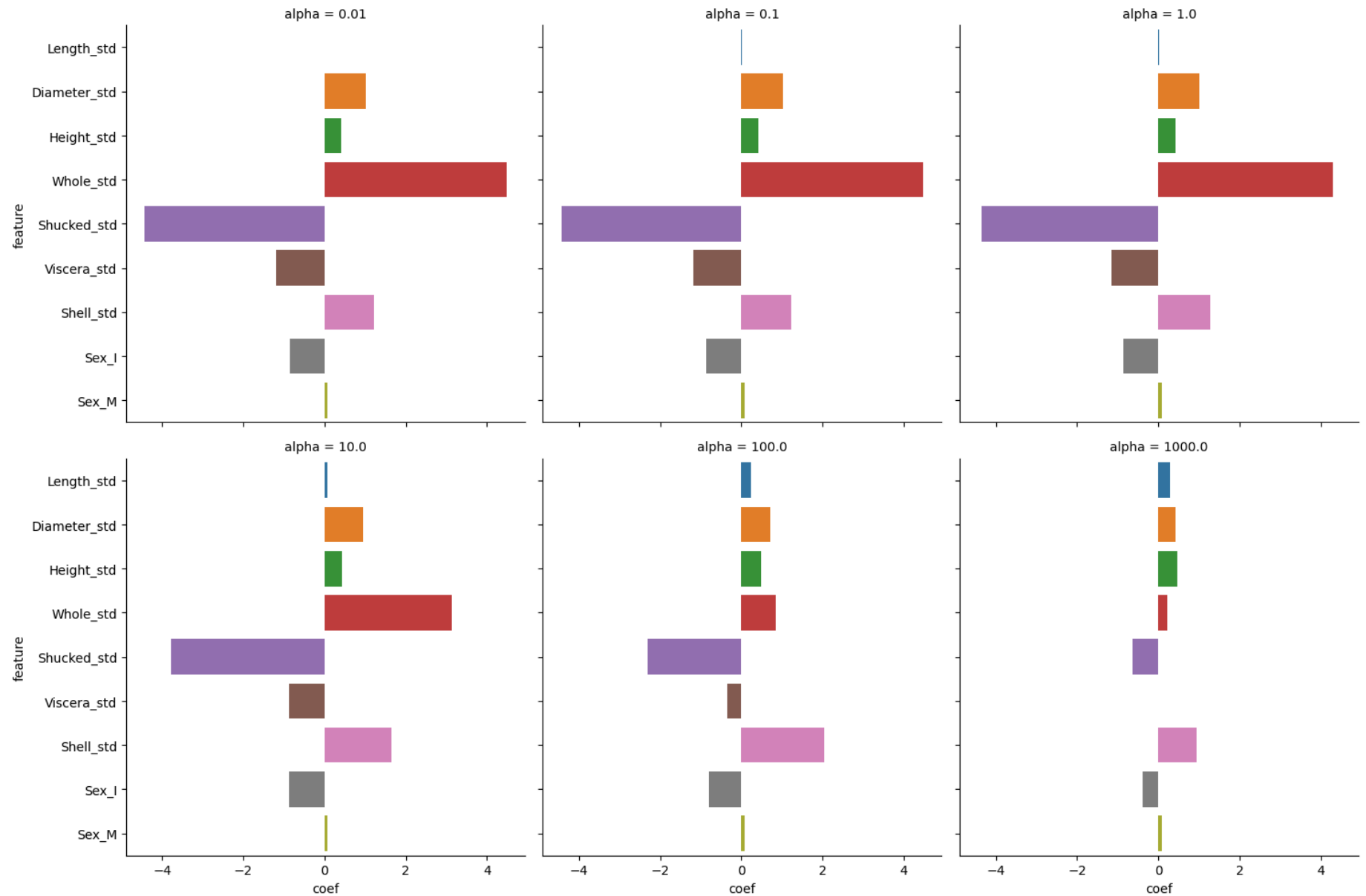
2. 시각화

- ``sns.catplot(data=df_coef, y='feature', x='coef', col='alpha', col_wrap=3, kind='bar')``:
- ``data=df_coef``:
앞에서 생성한 데이터프레임 ``df_coef``를 사용합니다.
- ``y='feature``:
y축에는 피처 이름을 표시합니다.
- ``x='coef``:
x축에는 회귀 계수를 표시합니다.
- ``col='alpha``:
각 규제 계수(알파 값)에 대해 별도의 그래프를 만듭니다.
- ``col_wrap=3``:
그래프를 3열로 정렬하여 래핑합니다.
- ``kind='bar``:
막대 그래프로 시각화합니다.

시각화의 결과

코드는 다양한 규제 계수(알파 값)에 대해 회귀 모델의 각 피처에 대한 계수를 시각화합니다. 결과 그래프는 여러 개의 서브플롯으로 구성되며, 각 서브플롯은 하나의 알파 값에 대응합니다. 각 서브플롯에서는 y축에 피처 이름, x축에 회귀 계수가 표시된 막대 그래프가 그려집니다. 이를 통해 규제 계수가 회귀 계수에 미치는 영향을 직관적으로 비교할 수 있습니다.

```
'''  
  
# 규제 계수별로 회귀 계수를 시각화합니다.  
df_coef  
    = pd.DataFrame(coefs, index=pd.Index(alphas, name='alpha'), \  
                   columns=X_ct_name)\  
       .reset_index()\  
       .melt(id_vars='alpha', value_name='coef', \  
            var_name='feature')  
  
sns.catplot(data=df_coef, y='feature', x='coef',  
            col='alpha', col_wrap=3, kind='bar')  
plt.show()
```

```
In [16]: '''
이 코드는 다양한 규제 계수(알파 값)에 따른 훈련 및
테스트 데이터의 평균 제곱 오차(MSE)를 시각화하고,
테스트 데이터의 MSE가 최소화되는 알파 값을
```

강조하는 작업을 수행합니다.

단계별 설명

1. x축의 로그 스케일 설정

- `plt.gca()`: 현재의 Axes 객체를 가져옵니다.
- `.set(xscale='log')`:
x축을 로그 스케일로 설정합니다.
이는 알파 값이 일반적으로 여러 배수로
변화하기 때문에 로그 스케일을 사용하여
더 직관적으로 표현합니다.

2. 훈련 데이터의 MSE 시각화

- `sns.lineplot(x = alphas, y = mse_train,
label='MSE of train')`:
훈련 데이터의 MSE를 선 그래프로 시각화합니다.
- `x = alphas`:
x축에 규제 계수(알파 값)들을 설정합니다.
- `y = mse_train`:
y축에 훈련 데이터에 대한 MSE 값을 설정합니다.
- `label='MSE of train'`:
범례에 표시될 레이블을 설정합니다.

3. 테스트 데이터의 MSE 시각화

- `sns.lineplot(x = alphas, y = mse_test,
label='MSE of test')`:
테스트 데이터의 MSE를 선 그래프로 시각화합니다.
- `x = alphas`:
x축에 규제 계수(알파 값)들을 설정합니다.
- `y = mse_test`:
y축에 테스트 데이터에 대한 MSE 값을 설정합니다.
- `label='MSE of test'`:
범례에 표시될 레이블을 설정합니다.

4. 최소 테스트 MSE에 대한 알파 값 강조

- `plt.axvline(x = alphas[np.argmin(mse_test)],
linestyle='dotted')`:
테스트 데이터의 MSE가 최소화되는
알파 값에서 수직선을 그립니다.
- `x = alphas[np.argmin(mse_test)]`:
`mse_test` 배열에서 최소값을 갖는 인덱스를
찾아 해당 인덱스에 대응하는 알파 값을 구합니다.
- `linestyle='dotted'`:
수직선의 스타일을 점선으로 설정합니다.

5. 범례 추가

- `plt.legend()`: 그래프에 범례를 추가하여 각 선이 무엇을 나타내는 지 설명합니다.

시각화의 결과

이 코드는 다양한 규제 계수(알파 값)에 대한 훈련 및 테스트 데이터의 MSE 변화를 시각적으로 비교할 수 있는 선 그래프를 생성합니다. 주요 특징은 다음과 같습니다:

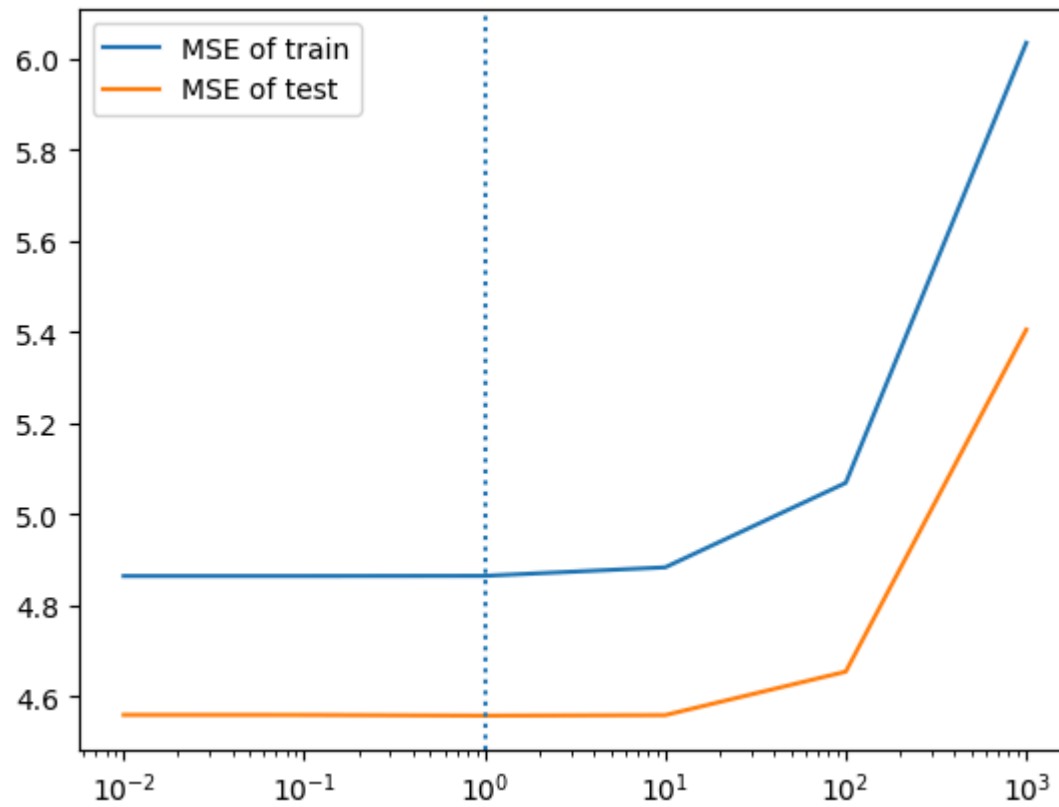
- 로그 스케일의 x축: 알파 값의 범위를 효과적으로 시각화합니다.
- 두 개의 선 그래프: 훈련 데이터와 테스트 데이터의 MSE 변화를 각각 나타냅니다.
- 점선 수직선: 테스트 데이터의 MSE가 최소화되는 알파 값을 강조합니다.
- 범례: 각 선의 의미를 명확히 합니다.

이 시각화를 통해 규제 계수가 모델의 성능에 미치는 영향을 직관적으로 이해할 수 있습니다. 특히, 테스트 데이터의 MSE가 최소화되는 지점을 확인함으로써

최적의 규제 계수를 선택하는 데 도움을 받을 수 있습니다.

'''

```
plt.gca().set(xscale='log')
sns.lineplot(x = alphas, y = mse_train, label='MSE of train')
sns.lineplot(x = alphas, y = mse_test, label='MSE of test')
plt.axvline(x = alphas[np.argmin(mse_test)], linestyle='dotted')
plt.legend()
plt.show()
```



In [2]: Image('159.png')

Out[2]: **Lasso**

- L1 규제를 손실함수에 더한 선형 회귀 모델입니다.

sklearn.linear_model.Lasso

- 손실 함수

$$L_{lasso}(y, \hat{y}) = \frac{1}{2n} \|y - \hat{y}\|_2^2 + \alpha \|\hat{\beta}\|_1$$

- 주요 하이퍼 파라미터

하이퍼 파라미터 설명

하이퍼 파라미터	설명
alpha	규제 계수

[Ex.4]

[Ex.2]에서 Ridge 모델대신 Lasso 모델의 규제 계수(alpha)에 따라

df_train과 df_test에 대한 평균제곱오차

(Mean Squared Error)를 비교해 봅니다.

```
alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 1e1]
```

In [17]:

```
'''
코드는 다양한 규제 계수(알파 값)에 대해
라쏘(Lasso) 회귀 모델을 학습시키고, 각 모델의
회귀 계수와 훈련 및 테스트 데이터의 평균 제곱 오차
(MSE)를 계산하여 저장하는 작업을 수행합니다.

### 단계별 설명
```

```

1. 필요한 모듈 및 변수 준비
- `from sklearn.linear_model import Lasso`:
    Scikit-learn에서 Lasso 회귀 모델을 임포트합니다.
- `alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 1e1]`:
    사용할 규제 계수(알파 값)들의 리스트를 정의합니다.
- `coefs, mse_train, mse_test = list(), list(), list()`:
    회귀 계수, 훈련 데이터 MSE, 테스트 데이터 MSE를
    저장할 빈 리스트를 초기화합니다.

2. 각 알파 값에 대해 라쏘 회귀 모델 학습 및 평가
- `for alpha in alphas`:
    각 알파 값에 대해 반복문을 실행합니다.
- `reg_ls = make_pipeline(ct, Lasso(alpha=alpha,
                                   random_state=123))`:
- `make_pipeline(ct, Lasso(alpha=alpha,
                           random_state=123))`:
    `ct`(컬럼 변환기)를 사용하여 데이터 전처리 후
    라쏘 회귀 모델을 생성하는 파이프라인을 만듭니다.
    여기서 `alpha`는 현재 반복에서 사용되는
    규제 계수입니다.
- `reg_ls.fit(df_abalone[X_cols], df_abalone['Rings'])`:
    주어진 데이터(`df_abalone`)의 독립 변수(`X_cols`)와
    종속 변수(`Rings`)를 사용하여 파이프라인을 학습시킵니다.
- `coefs.append(reg_ls[1].coef_)`:
    학습된 라쏘 모델의 회귀 계수를 리스트 `coefs`에
    추가합니다. `reg_ls[1]`은 파이프라인의
    두 번째 단계인 `Lasso` 모델을 의미합니다.
- `mse_train.append(
    mean_squared_error(df_train['Rings'],
                       reg_ls.predict(df_train[X_cols])))`:
    훈련 데이터에 대한 예측 값을 구하고,
    이를 사용하여 MSE를 계산한 후
    리스트 `mse_train`에 추가합니다.
- `mse_test.append(
    mean_squared_error(df_test['Rings'],
                      reg_ls.predict(df_test[X_cols])))`:
    테스트 데이터에 대한 예측 값을 구하고,
    이를 사용하여 MSE를 계산한 후
    리스트 `mse_test`에 추가합니다.

3. 회귀 계수 데이터프레임 생성 및 변환
- `pd.DataFrame(coefs, index=pd.Index(alphas, name='alpha'),
                columns=X_ct_name)`:

```

- ``coefs``: 이전 단계에서 저장한 회귀 계수 리스트입니다.
- ``alphas``: 알파 값의 리스트입니다.
- ``X_ct_name``: 피쳐 이름들의 리스트입니다.
- 이 줄은 회귀 계수 리스트를 데이터프레임 ``df_coef``로 변환합니다. 인덱스는 알파 값이고, 열 이름은 피쳐 이름입니다. 인덱스 이름은 ``'alpha'``로 설정됩니다.
- ``reset_index()``: 인덱스를 열로 변환하여 데이터프레임을 재구성합니다.
- ``melt(id_vars='alpha', value_name='coef', var_name='feature')``: 데이터프레임을 긴 형식으로 변환합니다.
- ``id_vars='alpha'``: ``'alpha'`` 열은 식별 변수로 유지됩니다.
- ``value_name='coef'``: 변환 후 값 열의 이름을 ``'coef'``로 설정합니다.
- ``var_name='feature'``: 변환 후 변수 이름을 ``'feature'``로 설정합니다.

최종적으로 ``df_coef``는 다음과 같은 형태를 가지게 됩니다:

```
alpha feature coef
0.0001 feature1 0.5
0.0001 feature2 1.2
0.0001 feature3 -0.3
...
```

코드의 전체적인 흐름은 다양한 알파 값에 대해 라쏘 회귀 모델을 학습시키고, 각 모델의 회귀 계수와 훈련 및 테스트 데이터의 MSE를 계산하여 이를 데이터프레임으로 정리하는 것입니다. 이 데이터프레임은 이후 시각화 등에 활용될 수 있습니다.

```
from sklearn.linear_model import Lasso

alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 1e1]
coefs, mse_train, mse_test = list(), list(), list()

for alpha in alphas:
    reg_ls = make_pipeline(ct,
                           Lasso(alpha=alpha,
```

```

                                random_state=123))
reg_ls.fit(df_abalone[X_cols], df_abalone['Rings'])
coefs.append(reg_ls[1].coef_)
mse_train.append(\
    mean_squared_error(df_train['Rings'], \
                        reg_ls.predict(df_train[X_cols])))

mse_test.append(\
    mean_squared_error(df_test['Rings'], \
                        reg_ls.predict(df_test[X_cols])))

df_coef \
= pd.DataFrame(coefs,
               index=pd.Index(alphas, name='alpha'),
               columns=X_ct_name)\
    .reset_index()\
    .melt(id_vars='alpha', value_name='coef',
          var_name='feature')

```

In [18]:

```
'''
코드는 `sns.catplot`을 사용하여 다양한 규제 계수(알파 값)에
따른 라쏘 회귀 모델의 회귀 계수를 시각화합니다.

### 코드 설명
1. 데이터 준비
- `data=df_coef`: `df_coef` 데이터프레임을 사용합니다.
  이 데이터프레임은 이전 단계에서 각 규제 계수(알파 값)별로
  변환된 회귀 계수 데이터를 포함합니다.
  데이터프레임은 다음과 같은 형태입니다:
  alpha  feature  coef
0.0001  feature1  0.5
0.0001  feature2  1.2
0.0001  feature3 -0.3
...

2. y축 설정
- `y='feature'`: y축에는 회귀 모델의 피쳐 이름을 표시합니다.
  각 막대는 하나의 피쳐를 나타냅니다.

3. x축 설정
- `x='coef'`: x축에는 피쳐의 회귀 계수를 표시합니다.
  막대의 길이는 각 피쳐의 회귀 계수 값을 나타냅니다.
'''

```


4. 컬럼 설정

- ``col='alpha'``: 각 알파 값(규제 계수)에 대해 별도의 그래프(서브플롯)를 만듭니다.
즉, 각 알파 값에 대해 하나의 서브플롯이 생성됩니다.

5. 그래프 배열

- ``col_wrap=3``: 서브플롯을 3개의 열로 배열합니다.
즉, 한 줄에 최대 3개의 서브플롯이 표시됩니다.
이 설정은 서브플롯이 너무 많이 나열되는 것을 방지하여 그래프를 더 읽기 쉽게 만듭니다.

6. 그래프 유형

- ``kind='bar'``: 막대 그래프로 시각화합니다.
각 서브플롯은 알파 값에 대한 막대 그래프입니다.

시각화의 결과

코드는 다양한 규제 계수(알파 값)에 따른 피처의 회귀 계수를 막대 그래프로 시각화합니다.

결과 그래프는 다음과 같은 특징을 가집니다:

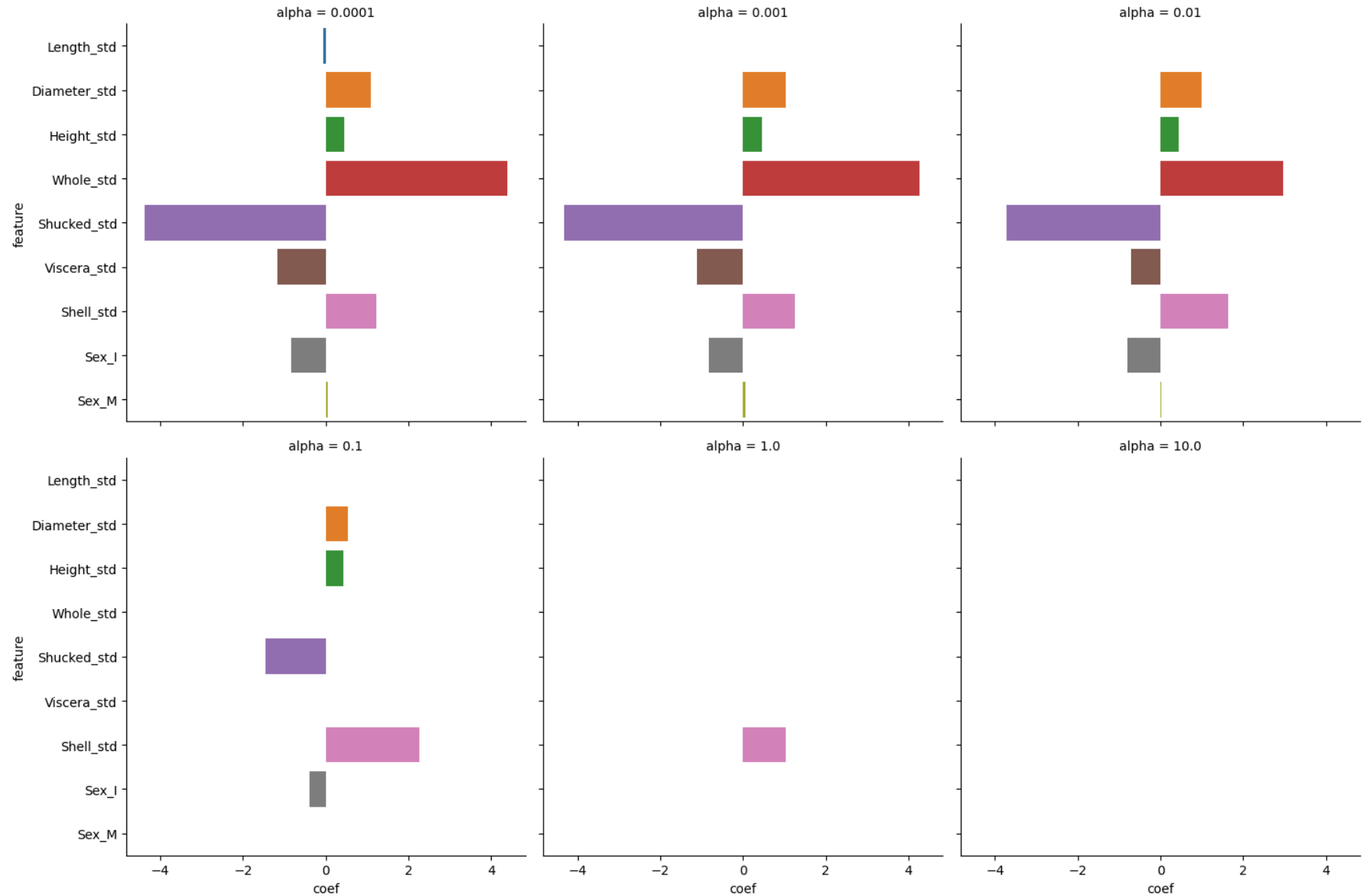
- 여러 서브플롯: 각 알파 값에 대해 하나의 서브플롯이 생성됩니다.
예를 들어, 알파 값이 `0.0001`인 서브플롯, `0.001`인 서브플롯, `0.01`인 서브플롯 등이 있습니다.
- 피처별 막대: 각 서브플롯에서는 y축에 피처 이름, x축에 해당 피처의 회귀 계수가 표시됩니다.
막대의 길이는 회귀 계수 값을 나타냅니다.
- 가독성: ``col_wrap=3`` 설정으로 인해 한 줄에 최대 3개의 서브플롯이 표시되어 전체 그래프의 가독성을 높입니다.

이 시각화를 통해 다양한 규제 계수에 따라 피처의 회귀 계수가 어떻게 변하는지를 직관적으로 비교할 수 있습니다.

이는 모델의 규제 강도가 피처 선택과 계수 크기에 미치는 영향을 시각적으로 이해하는 데 도움이 됩니다.

```
'''
```

```
sns.catplot(data=df_coef, y='feature', x='coef', \
            col='alpha', col_wrap=3, kind='bar')
plt.show()
```



```
In [19]: '''
코드는 다양한 규제 계수(알파 값)에 따른
훈련 및 테스트 데이터의 평균 제곱 오차(MSE)를
시각화하고, 테스트 데이터의 MSE가 최소화되는
```

알파 값을 강조하는 작업을 수행합니다.

1. x축의 로그 스케일 설정

- `plt.gca()`: 현재의 Axes 객체를 가져옵니다.
 - `.set(xscale='log')`: x축을 로그 스케일로 설정합니다.
- 이는 알파 값이 여러 배수로 변화하기 때문에 로그 스케일을 사용하여 더 직관적으로 표현합니다.

2. 훈련 데이터의 MSE 시각화

- `sns.lineplot(x = alphas, y = mse_train, label='MSE of train')`:
훈련 데이터의 MSE를 선 그래프로 시각화합니다.
- `x = alphas`:
x축에 규제 계수(알파 값)들을 설정합니다.
- `y = mse_train`:
y축에 훈련 데이터에 대한 MSE 값을 설정합니다.
- `label='MSE of train'`:
범례에 표시될 레이블을 설정합니다.

3. 테스트 데이터의 MSE 시각화

- `sns.lineplot(x = alphas, y = mse_test, label='MSE of test')`:
테스트 데이터의 MSE를 선 그래프로 시각화합니다.
- `x = alphas`:
x축에 규제 계수(알파 값)들을 설정합니다.
- `y = mse_test`:
y축에 테스트 데이터에 대한 MSE 값을 설정합니다.
- `label='MSE of test'`:
범례에 표시될 레이블을 설정합니다.

4. 최소 테스트 MSE에 대한 알파 값 강조

- `plt.axvline(x = alphas[np.argmin(mse_test)], linestyle='dotted')`:
테스트 데이터의 MSE가 최소화되는 알파 값에서 수직선을 그립니다.
- `x = alphas[np.argmin(mse_test)]`:
`mse_test` 배열에서 최소값을 갖는 인덱스를 찾아 해당 인덱스에 대응하는 알파 값을 구합니다.
- `linestyle='dotted'`:
수직선의 스타일을 점선으로 설정합니다.

5. 범례 추가

- `plt.legend()`: 그래프에 범례를 추가하여 각 선이 무엇을 나타내는지 설명합니다.

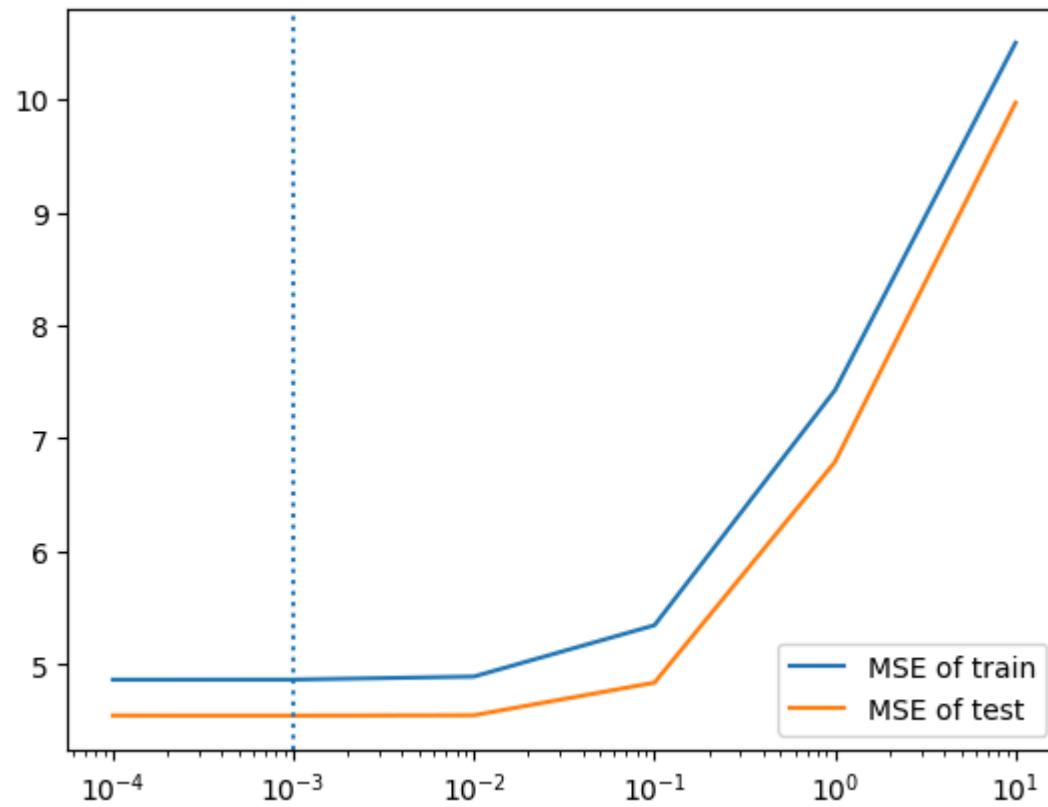
시각화의 결과

코드는 다양한 규제 계수(알파 값)에 대한 훈련 및 테스트 데이터의 MSE 변화를 시각적으로 비교할 수 있는 선 그래프를 생성합니다. 주요 특징은 다음과 같습니다:

- 로그 스케일의 x축:
알파 값의 범위를 효과적으로 시각화합니다.
- 두 개의 선 그래프:
훈련 데이터와 테스트 데이터의 MSE 변화를 각각 나타냅니다.
- 점선 수직선:
테스트 데이터의 MSE가 최소화되는 알파 값을 강조합니다.
- 범례: 각 선의 의미를 명확히 합니다.

이 시각화를 통해 규제 계수가 모델의 성능에 미치는 영향을 직관적으로 이해할 수 있습니다. 특히, 테스트 데이터의 MSE가 최소화되는 지점을 확인함으로써 최적의 규제 계수를 선택하는 데 도움을 받을 수 있습니다.

```
plt.gca().set(xscale='log')
sns.lineplot(x = alphas, y = mse_train, label='MSE of train')
sns.lineplot(x = alphas, y = mse_test, label='MSE of test')
plt.axvline(x = alphas[np.argmin(mse_test)], linestyle='dotted')
plt.legend()
plt.show()
```



In [2]: Image('160.png')

Out[2]: **Elastic net**

- L1, L2 규제를 손실함수에 더한 선형 회귀 모델입니다.

sklearn.linear_model.ElasticNet

- 손실 함수

$$L_{ElasticNet}(y, \hat{y}) = \frac{1}{2n} ||y - \hat{y}||_2^2 + \rho \cdot \alpha \cdot ||\hat{\beta}||_1 + 0.5(1 - \rho) \cdot \alpha \cdot ||\hat{\beta}||_2$$

ρ : l1_ratio

- 주요 하이퍼 파라미터

하이퍼 파라미터 설명

l1_ratio	l1 규제의 비율
----------	-----------

[Ex.5]

[Ex.2]에서 Ridge 모델대신 Elastic net 모델의 규제 계수(alpha)에

따라 df_train과 df_test에 대한

평균제곱오차(Mean Squared Error)를 비교해 봅니다.

이 때, l1_ratio는 0.5로 설정합니다.

alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 1e1]

In [20]: `'''`
 코드는 다양한 규제 계수(알파 값)에 대해

ElasticNet 회귀 모델을 학습시키고, 각 모델의 회귀 계수와 훈련 및 테스트 데이터의 평균 제곱 오차(MSE)를 계산하여 저장하는 작업을 수행합니다.

단계별 설명

1. 필요한 모듈 및 변수 준비

```
- `alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 1e1]`:  
  사용할 규제 계수(알파 값)들의 리스트를 정의합니다.  
- `coefs, mse_train, mse_test`  
  = list(), list(), list():  
  회귀 계수, 훈련 데이터 MSE, 테스트 데이터  
  MSE를 저장할 빈 리스트를 초기화합니다.
```

2. 각 알파 값에 대해 ElasticNet 회귀 모델 학습 및 평가

```
- `for alpha in alphas`:  
  각 알파 값에 대해 반복문을 실행합니다.  
- `reg_ls = make_pipeline(ct,  
  ElasticNet(alpha=alpha, l1_ratio=0.5,  
  random_state=123))`:  
- `make_pipeline(ct, ElasticNet(alpha=alpha,  
  l1_ratio=0.5, random_state=123))`:  
  `ct`(컬럼 변환기)를 사용하여 데이터 전처리 후  
  ElasticNet 회귀 모델을 생성하는  
  파이프라인을 만듭니다. 여기서 `alpha`는  
  현재 반복에서 사용되는 규제 계수입니다.  
  `l1_ratio=0.5`는 L1과 L2 규제의 비율을 설정합니다.  
- `reg_ls.fit(df_abalone[X_cols], df_abalone['Rings'])`:  
  주어진 데이터(`df_abalone`)의 독립 변수(`X_cols`)와  
  종속 변수(`Rings`)를 사용하여 파이프라인을 학습시킵니다.  
- `coefs.append(reg_ls[1].coef_)`:  
  학습된 ElasticNet 모델의 회귀 계수를  
  리스트 `coefs`에 추가합니다.  
  `reg_ls[1]`은 파이프라인의 두 번째 단계인  
  `ElasticNet` 모델을 의미합니다.  
- `mse_train.append(mean_squared_error(df_train['Rings'],  
  reg_ls.predict(df_train[X_cols])))`:  
  훈련 데이터에 대한 예측 값을 구하고, 이를 사용하여  
  MSE를 계산한 후 리스트 `mse_train`에 추가합니다.
```

```
- `mse_test.append(mean_squared_error(df_test['Rings'],
    reg_ls.predict(df_test[X_cols])))`:
    테스트 데이터에 대한 예측 값을 구하고, 이를 사용하여
    MSE를 계산한 후 리스트 `mse_test`에 추가합니다.
```

3. 회귀 계수 데이터프레임 생성 및 변환

```
- `pd.DataFrame(coefs, index=pd.Index(alphas, name='alpha'),
    columns=X_ct_name)`:
    - `coefs`: 이전 단계에서 저장한 회귀 계수 리스트입니다.
    - `alphas`: 알파 값의 리스트입니다.
    - `X_ct_name`: 피쳐 이름들의 리스트입니다.
    - 이 줄은 회귀 계수 리스트를 데이터프레임
      `df_coef`로 변환합니다. 인덱스는 알파 값이고,
      열 이름은 피쳐 이름입니다.
      인덱스 이름은 `'alpha'`로 설정됩니다.
    - `.reset_index()`:
      인덱스를 열로 변환하여 데이터프레임을 재구성합니다.
    - `.melt(id_vars='alpha', value_name='coef',
      var_name='feature')`:
      데이터프레임을 긴 형식으로 변환합니다.
    - `id_vars='alpha'`:
      `'alpha'` 열은 식별 변수로 유지됩니다.
    - `value_name='coef'`:
      변환 후 값 열의 이름을 `'coef'`로 설정합니다.
    - `var_name='feature'`:
      변환 후 변수 이름을 `'feature'`로 설정합니다.
```

최종적으로 `df\_coef`는
다음과 같은 형태를 가지게 됩니다:

```
alpha  feature  coef
0.0001  feature1  0.5
0.0001  feature2  1.2
0.0001  feature3 -0.3
...
```

코드의 전체적인 흐름은 다양한 알파 값에 대해
ElasticNet 회귀 모델을 학습시키고,
각 모델의 회귀 계수와 훈련 및 테스트 데이터의
MSE를 계산하여 이를 데이터프레임으로 정리하는 것입니다.
이 데이터프레임은 이후 시각화 등에 활용될 수 있습니다.
...


```

from sklearn.linear_model import ElasticNet

alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 1e1]
coefs, mse_train, mse_test = list(), list(), list()

for alpha in alphas:
    reg_ls = make_pipeline(ct,
                           ElasticNet(alpha=alpha,
                                       l1_ratio=0.5,
                                       random_state=123))
    reg_ls.fit(df_abalone[X_cols],
               df_abalone['Rings'])
    coefs.append(reg_ls[1].coef_)
    mse_train.append(\
        mean_squared_error(df_train['Rings'],
                           reg_ls.predict(df_train[X_cols])))
    mse_test.append(\
        mean_squared_error(df_test['Rings'],
                           reg_ls.predict(df_test[X_cols])))

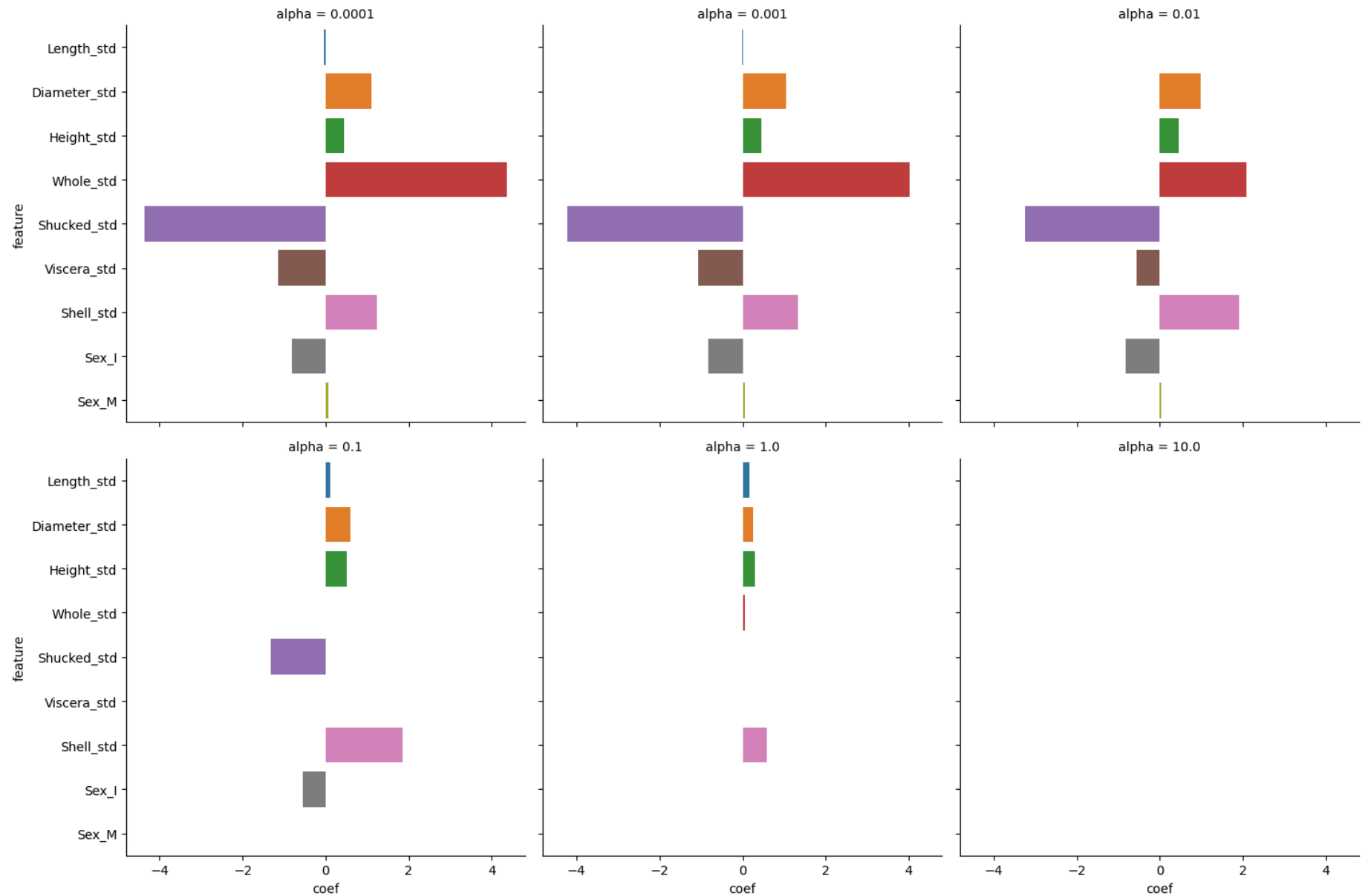
df_coef
= pd.DataFrame(coefs, index=pd.Index(alphas, \
                                     name='alpha'), \
               columns=X_ct_name)\
    .reset_index()\
    .melt(id_vars='alpha', value_name='coef',
         var_name='feature')

```

```

In [21]: sns.catplot(data=df_coef, y='feature', x='coef',
                    col='alpha', col_wrap=3, kind='bar')
plt.show()

```

**[Ex.6]**

[Ex.5]를 기반으로 규제 계수(α)는 1로 고정하고,

아래 박스의 `l1_ratios` 들의 `df_train`과 `df_test`에 대한
평균제곱오차(Mean Squared Error)를 비교해 봅니다.

```
l1_ratios = [0.0, 0.2, 0.4, 0.6, 0.8, 1]
```

```
In [ ]: '''
코드는 다양한 L1 비율(l1_ratio)에 대해
ElasticNet 회귀 모델을 학습시키고,
각 모델의 회귀 계수와 훈련 및 테스트 데이터의
평균 제곱 오차(MSE)를 계산하여 저장하는 작업을 수행합니다.
`alpha` 값은 고정되어 있고, L1 비율을 변화시키면서
모델을 평가합니다.

1. 필요한 모듈 및 변수 준비
- `l1_ratios = [0.0, 0.2, 0.4, 0.6, 0.8, 1]`:
  사용할 L1 비율들의 리스트를 정의합니다.
- `coefs, mse_train, mse_test = list(), list(), list()`:
  회귀 계수, 훈련 데이터 MSE, 테스트 데이터 MSE를
  저장할 빈 리스트를 초기화합니다.
- `alpha = 1`: 고정된 규제 계수(알파 값)를 설정합니다.

2. 각 L1 비율에 대해 ElasticNet 회귀 모델 학습 및 평가
- `for l1_ratio in l1_ratios`:
  각 L1 비율에 대해 반복문을 실행합니다.
- `reg_ls = make_pipeline(ct, ElasticNet(alpha=alpha,
  l1_ratio=l1_ratio))`:
- `make_pipeline(ct, ElasticNet(alpha=alpha,
  l1_ratio=l1_ratio))`:
  `ct`(컬럼 변환기)를 사용하여 데이터 전처리 후
  ElasticNet 회귀 모델을 생성하는 파이프라인을 만듭니다.
  여기서 `alpha`는 고정된 값이고,
  `l1_ratio`는 현재 반복에서 사용되는 값입니다.
- `reg_ls.fit(df_abalone[X_cols],
  df_abalone['Rings'])`:
  주어진 데이터(`df_abalone`)의 독립 변수(`X_cols`)와
  종속 변수(`Rings`)를 사용하여 파이프라인을 학습시킵니다.
- `coefs.append(reg_ls[1].coef_)`:
```

```

        학습된 ElasticNet 모델의 회귀 계수를 리스트
        `coefs`에 추가합니다.
        `reg_ls[1]`은 파이프라인의 두 번째 단계인
        `ElasticNet` 모델을 의미합니다.
- `mse_train.append(mean_squared_error(df_train['Rings'],
                                         reg_ls.predict(df_train[X_cols])))`:
    훈련 데이터에 대한 예측 값을 구하고, 이를 사용하여
    MSE를 계산한 후 리스트 `mse_train`에 추가합니다.
- `mse_test.append(mean_squared_error(
    df_test['Rings'], reg_ls.predict(df_test[X_cols])))`:
    테스트 데이터에 대한 예측 값을 구하고,
    이를 사용하여 MSE를 계산한 후 리스트
    `mse_test`에 추가합니다.
3. 회귀 계수 데이터프레임 생성 및 변환
- `pd.DataFrame(coefs, index=pd.Index(l1_ratios,
    name='l1_ratio'), columns=X_ct_name)`:
- `coefs`: 이전 단계에서 저장한 회귀 계수 리스트입니다.
- `l1_ratios`: L1 비율들의 리스트입니다.
- `X_ct_name`: 피처 이름들의 리스트입니다.
- 이 줄은 회귀 계수 리스트를 데이터프레임 `df_coef`로
    변환합니다. 인덱스는 L1 비율이고,
    열 이름은 피처 이름입니다.
    인덱스 이름은 `l1_ratio`로 설정됩니다.
- `.reset_index()`:
    인덱스를 열로 변환하여 데이터프레임을 재구성합니다.
- `.melt(id_vars='l1_ratio', value_name='coef',
    var_name='feature')`:
    데이터프레임을 긴 형식으로 변환합니다.
- `id_vars='l1_ratio'`:
    `l1_ratio` 열은 식별 변수로 유지됩니다.
- `value_name='coef'`:
    변환 후 값 열의 이름을 `coef`로 설정합니다.
- `var_name='feature'`:
    변환 후 변수 이름을 `feature`로 설정합니다.

최종적으로 `df_coef`는 다음과 같은 형태를 가지게 됩니다:
l1_ratio  feature  coef
0.0       feature1  0.5
0.0       feature2  1.2
0.0       feature3 -0.3
...

```

```

코드의 전체적인 흐름은 다양한 L1 비율에 대해 ElasticNet
회귀 모델을 학습시키고, 각 모델의 회귀 계수와 훈련 및
테스트 데이터의 MSE를 계산하여 이를 데이터프레임으로
정리하는 것입니다. 이 데이터프레임은 이후
시각화 등에 활용될 수 있습니다.
'''

from sklearn.linear_model import ElasticNet

l1_ratios = [0.0, 0.2, 0.4, 0.6, 0.8, 1]
coefs, mse_train, mse_test = list(), list(), list()
alpha = 1
for l1_ratio in l1_ratios:
    reg_ls \
        = make_pipeline(ct, ElasticNet(alpha=alpha,
                                       l1_ratio=l1_ratio))
    reg_ls.fit(df_abalone[X_cols], df_abalone['Rings'])
    coefs.append(reg_ls[1].coef_)
    mse_train.append(\
        mean_squared_error(df_train['Rings'],
                           reg_ls.predict(df_train[X_cols])))

    mse_test.append(\
        mean_squared_error(df_test['Rings'],
                           reg_ls.predict(df_test[X_cols])))

df_coef = pd.DataFrame(coefs,
                       index=pd.Index(l1_ratios,
                                       name='l1_ratio'),
                       columns=X_ct_name)\
    .reset_index().melt(id_vars='l1_ratio',\
                       value_name='coef', \
                       var_name='feature')

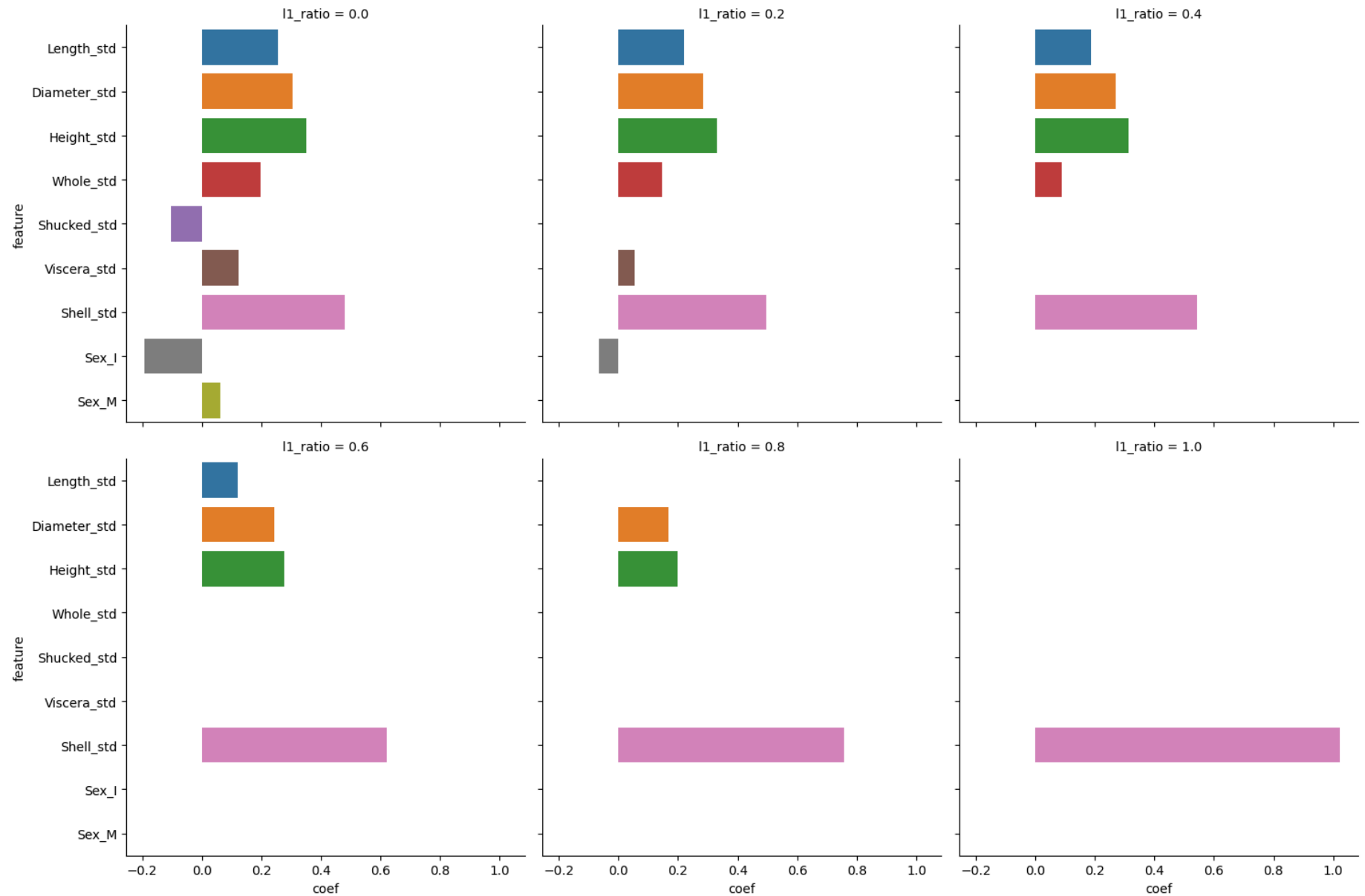
```

In [23]: '''
 코드는 `seaborn` 라이브러리를 사용하여 `catplot`을 통해
 막대 그래프(bar plot)를 생성하는 기능을 수행합니다.
 `df\_coef` 데이터프레임을 사용하여 특정 변수들의 계수를
 시각화하는데, `l1\_ratio`라는 변수에 따라 여러 개의
 그래프를 생성합니다. 각 그래프는 특정 `l1\_ratio` 값에
 해당하는 데이터를 보여줍니다.

1. `sns.catplot``:
Seaborn 라이브러리의 함수로, 범주형 데이터의 시각화를 돕는 함수입니다. 다양한 그래프 유형을 지원하며, 이 코드에서는 막대 그래프를 그리도록 설정되어 있습니다.
2. `data=df_coef``:
`catplot`` 함수에 사용될 데이터프레임을 지정합니다. 여기서는 `df_coef``라는 이름의 데이터프레임이 사용됩니다.
3. `y='feature``:
y축에 사용할 변수를 지정합니다.
`df_coef`` 데이터프레임에서 `feature`` 열의 값이 y축에 나타납니다. 이는 시각화하려는 특성(feature)을 의미합니다.
4. `x='coef``:
x축에 사용할 변수를 지정합니다.
`df_coef`` 데이터프레임에서 `coef`` 열의 값이 x축에 나타납니다. 이는 각 특성의 계수(coefficient) 값을 의미합니다.
5. `col='l1_ratio``:
그래프를 나눌 기준 열을 지정합니다.
`df_coef`` 데이터프레임의 `l1_ratio`` 열을 기준으로 데이터가 분할되어 별도의 그래프가 생성됩니다.
6. `col_wrap=3``:
그래프를 몇 개의 열로 나눌지를 지정합니다.
여기서는 각 `l1_ratio`` 값에 대한 그래프를 한 줄에 3개씩 배치합니다. 예를 들어, 6개의 `l1_ratio`` 값이 있다면, 2줄에 걸쳐 3개의 그래프가 각각 배치됩니다.
7. `kind='bar``:
생성할 그래프의 유형을 지정합니다. 여기서는 막대 그래프(bar plot)를 그리도록 설정되어 있습니다.

따라서, 이 코드는 `df_coef`` 데이터프레임에 있는 각 `feature``의 `coef`` 값을 시각화하는 막대 그래프를 여러 개 생성합니다. 각 그래프는 서로 다른 `l1_ratio`` 값에 대해 분할되어 있으며, 한 줄에 3개의 그래프가 배치됩니다. 이러한 시각화는 데이터가 `l1_ratio`` 값에 따라 어떻게 변화하는지를 한눈에 비교하기 쉽게 만듭니다.

```
'''  
sns.catplot(data=df_coef, y='feature', x='coef',  
            col='l1_ratio', col_wrap=3, kind='bar')  
plt.show()
```



```
In [24]: '''
코드는 `seaborn`과 `matplotlib` 라이브러리를
사용하여 선 그래프(line plot)를 그리는 코드입니다.
이 그래프는 `l1_ratios` 값을 x축으로 하고,
```


`mse\_train`과 `mse\_test` 값을 y축으로 하여,
 훈련 데이터와 테스트 데이터에 대한 평균 제곱 오차
 (Mean Squared Error, MSE)를 시각화합니다.
 또한, 테스트 데이터의 MSE가 최소가 되는 지점에
 수직선을 추가합니다.

```
`plt.axvline(x = alphas[np.argmin(mse_test)],
             linestyle='dotted')`:
```

- `matplotlib.pyplot`의 `axvline` 함수를 사용하여 수직선을 추가합니다.
- `x` 인자에 `alphas[np.argmin(mse\_test)]` 값을 지정하여 수직선이 그려질 x축의 위치를 설정합니다.
- `np.argmin(mse\_test)`는 `mse\_test` 배열에서 최소값의 인덱스를 반환합니다.
- `alphas` 배열에서 해당 인덱스에 있는 값을 가져와 x축의 위치로 사용합니다.
- `linestyle='dotted'` 인자를 사용하여 수직선의 스타일을 점선으로 설정합니다.

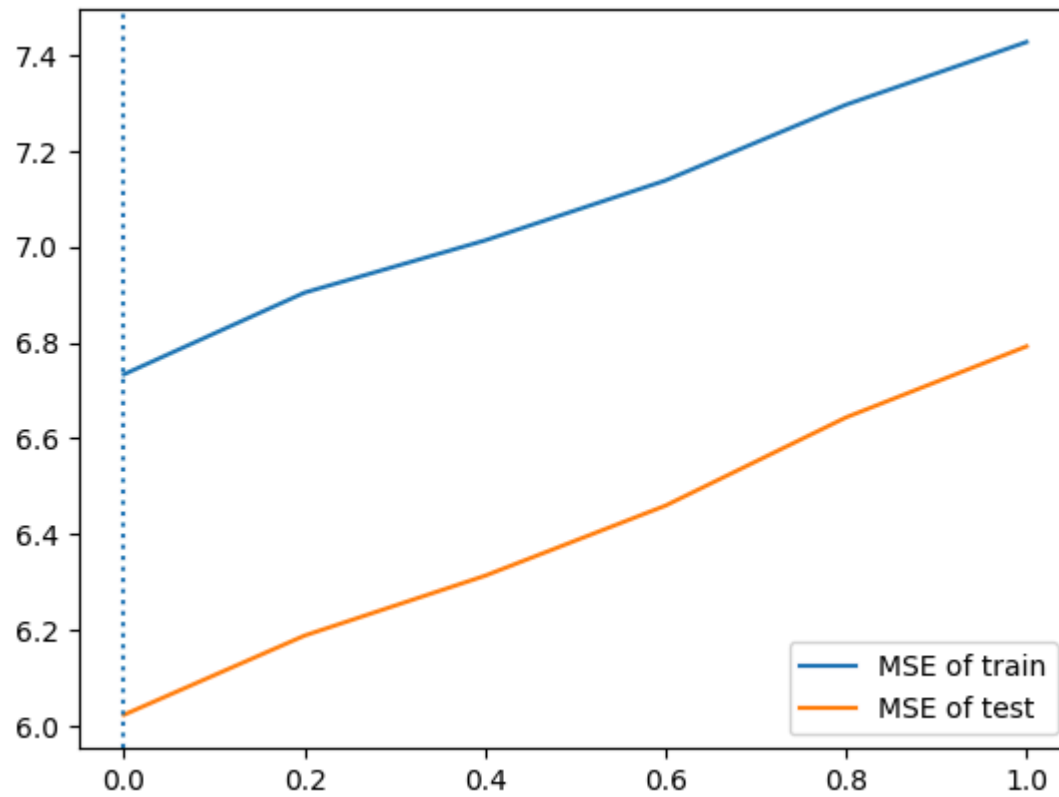
전체적인 동작 설명:

- `l1\_ratios` 값에 따라 `mse\_train`과 `mse\_test` 값을 각각 선 그래프로 그립니다.
- 테스트 데이터의 MSE가 최소가 되는 지점에 점선으로 수직선을 추가하여, 최적의 `l1\_ratio` 값을 시각적으로 표시합니다.

이 코드의 목적은 `l1\_ratios` 값에 따른 훈련 및 테스트 데이터의 평균 제곱 오차를 비교하고,
 최적의 `l1\_ratio` 값을 확인하는 것입니다.

```
'''
```

```
sns.lineplot(x = l1_ratios, y = mse_train, label='MSE of train')
sns.lineplot(x = l1_ratios, y = mse_test, label='MSE of test')
plt.axvline(x = alphas[np.argmin(mse_test)], linestyle='dotted')
plt.legend()
plt.show()
```



In [2]: Image('161.png')

Out[2]: 2. 로지스틱 회귀 모델(Logistic Regression Model)

이진 분류

- 링크 함수: 선형 회귀 모델의 출력의 스케일을 바꾸어주는 함수입니다.

선형회귀 모델의 출력은 $[-\infty, \infty]$ 이고 대상 변수의 영역은 $[0, 1]$ 입니다.

만일 선형 회귀 모델을 분류 문제에 그대로 사용하게 한다면 출력이 음(Negative)의 클래스는 0을 중심으로 양(Positive)의 클래스는 1을 중심으로 출력이 모이도록 분포해야 합니다. $[-\infty, \infty]$ 를 범위로 하고 있어 구조상 최적의 해를 찾기 쉽지 않습니다.

선형회귀 모델이 음으로 갈수록 음(Negative)의 클래스 양수로 갈 수록 양(Positive) 클래스일 정도를 나타내게 하게 하도록 스케일을 변경시켜준다면, 최적의 해를 보다 찾기 쉬워지고 출력을 직관적으로 이해하기 쉽게 해줍니다.

- 로짓(Logit) 함수: 확률의 영역 $[0, 1]$ 의 수를 $[-\infty, \infty]$ 으로 바꾸어 주는 함수 입니다.

$odd(\hat{y}) = \frac{\hat{y}}{1-\hat{y}}$: $[0, 1]$ 의 수를 $[0, \infty]$ 로 바꾸어 줍니다.

$logit(\hat{y}) = \log(\frac{\hat{y}}{1-\hat{y}})$: $[0, 1]$ 의 수를 $[-\infty, \infty]$ 로 바꾸어 줍니다.

- 로지스틱(Logistic) 함수: $[-\infty, \infty]$ 의 수를 $[0, 1]$ 의 수로 바꾸어 줍니다. Logit의 역함수입니다.

$$x = \log(\frac{\hat{y}}{1-\hat{y}})$$

$$e^x = \frac{\hat{y}}{1-\hat{y}}$$

$$(1 - \hat{y})e^x = \hat{y}$$

$$(1 + e^x)\hat{y} = e^x$$

$$\hat{y} = \frac{e^x}{1+e^x} = \frac{1}{1+e^{-x}}$$

- 선형회귀 모델에서 링크 함수로 로지스틱(Logistic) 함수를 사용하는 선형 회귀 모델을 로지스틱 회귀 모델이라 합니다.
- 이진 크로스 엔트로피(Binary Cross Entropy): 이진 분류의 손실 함수입니다.

$$L_{\text{Logistic}}(y, \hat{y}) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

```
In [25]: '''
코드는 `matplotlib`과 `scipy` 라이브러리를 사용하여
로지스틱 함수의 그래프를 그리는 코드입니다.
로지스틱 함수는 S자 형태의 곡선을 가지며,
종종 시그모이드 함수(sigmoid function)라고도 불립니다.
이 함수는 분류 모델에서 확률을 예측하는 데 사용됩니다.

`from scipy.special import expit`:
- `expit` 함수는 로지스틱 함수
   $\sigma(x) = \frac{1}{1 + e^{-x}}$ 를 계산합니다.

`plt.plot(x, expit(x))`:
- x축에는 `x` 배열의 값을 사용하고,
  y축에는 `expit(x)` 값을 사용합니다.
- `expit(x)`는 `x`의 각 값에 대해
  로지스틱 함수를 적용한 결과입니다.

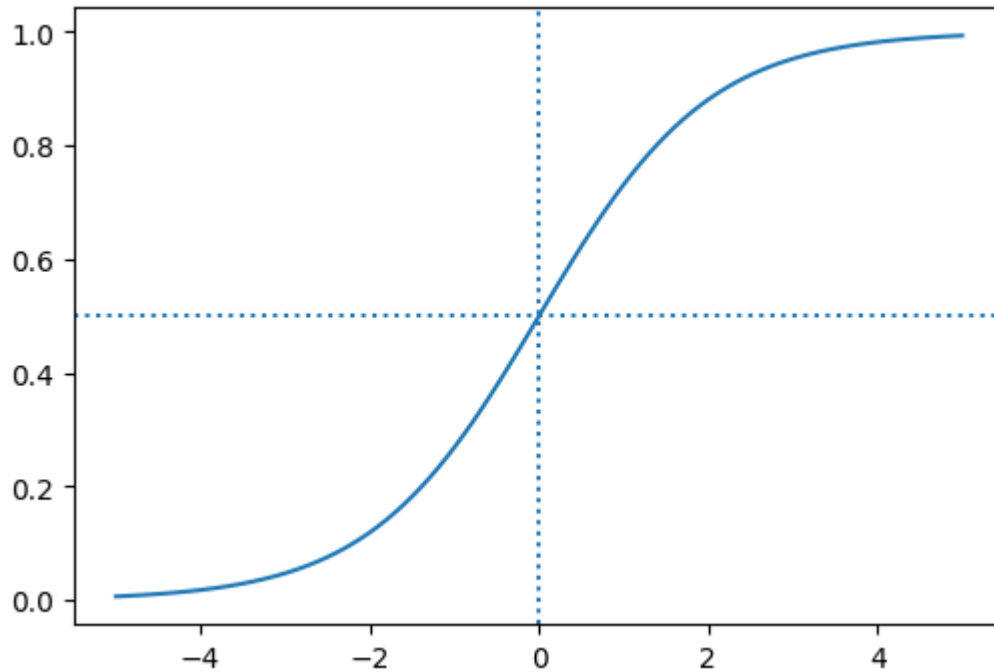
### 전체적인 동작 설명:
- x값이 -5에서 5까지 범위에서
  로지스틱 함수의 값을 계산하여 그래프로 그립니다.
- x=0 위치에 점선 수직선을 추가하여,
  로지스틱 함수가 0인 지점을 강조합니다.
- y=0.5 위치에 점선 수평선을 추가하여,
  로지스틱 함수 값이 0.5인 지점을 강조합니다.

이 코드의 목적은 로지스틱 함수의 형태를 시각화하여
함수의 특성과 중요한 지점을 명확하게 보여주는 것입니다.
로지스틱 함수는 -5에서 5까지의 범위에서
빠르게 0에서 1로 변하는 S자 형태의 곡선을 가지며,
x=0일 때 y=0.5를 가집니다.
'''

from scipy.special import expit

plt.figure(figsize=(6, 4))
```

```
x = np.linspace(-5, 5, 101)
plt.plot(x, expit(x))
plt.axvline(0, linestyle='dotted')
plt.axhline(0.5, linestyle='dotted')
plt.show()
```



In [3]: Image('162.png')

Out[3]: 다중 클래스 분류

- Softmax 함수: 다중 클래스 분류의 문제에서 클래스 수만큼의 로짓(Logit)에서 확률로 변환해주는 함수입니다.

$$\text{softmax}(p) = \frac{e^{p_i}}{\sum_{i=1}^c e^{p_i}}, c: \text{클래스의 수}$$

- 크로스 엔트로피(Cross Entropy): 다중 분류의 손실 함수입니다.

$$L_{Logistic}(y, \hat{y}) = - \sum_{i=1}^c y_i \log(\hat{y}_i),$$

y_i : 대상 변수 y 가 i 클래스 여부를 나타내는 벡터, $\hat{y}_i$: i 클래스일 확률 예측을 나타내는 벡터

sklearn.linear_model.LogisticRegression

$$L(y, \hat{y}) = C \sum_{i=1}^n s_i \left(-y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i) \right) + r(\hat{\beta})$$

penalty	r(w)
None	0
l_1	$ \hat{\beta} _1$
l_2	$\frac{1}{2} \hat{\beta} _2^2$
ElasticNet	$\frac{1-\rho}{2} \hat{\beta} _2^2 + \rho \hat{\beta} _1$

- 주요 하이퍼 파라미터

하이퍼 파라미터 설명

penalty None, l1, l2, ElasticNet

C 규제 강도의 역수(Inverse of regularization Strength), Capacity

multi_class 'auto', 'ovr', 'multinomial'
 ovr: One vs Rest
 multinomial: Softmax를 Loss로 합니다.

[Ex.7]

df_train가 df_test에서 성별(Sex)이 'F', 'M' 인 경우만 뽑아

df_train2, df_test2셋을 만들어,

성별(Sex)을 맞추는 로지스틱 회귀 모델을 만들어 df_train2로 학습을 시키고,

df_test2로 성능을 측정합니다.

성능 측정은 AUC(Area under of receiver characteristic Curve)로 합니다.

In [26]:

```
'''
코드는 `pandas` 라이브러리를 사용하여 특정 조건에 맞는
데이터프레임을 필터링하고, 그 결과로 얻은 새로운
데이터프레임에서 특정 열을 선택하는 과정을 수행합니다.

`df_train2 = df_train.loc[df_train['Sex'].isin(['F', 'M'])]`:
- `df_train` 데이터프레임에서 'Sex' 열의 값이
  'F' 또는 'M'인 행만 선택하여 `df_train2` 라는
  새로운 데이터프레임을 만듭니다.
- `df_train['Sex'].isin(['F', 'M'])`는 `df_train`
  데이터프레임의 'Sex' 열이 'F' 또는 'M'인지 여부를
  나타내는 불리언 시리즈를 반환합니다.
- `df_train.loc[...]`은 이 불리언 시리즈가 `True`인 행만 선택합니다.

`df_test2 = df_test.loc[df_test['Sex'].isin(['F', 'M'])]`:
- 위와 동일한 방식으로 `df_test` 데이터프레임에서
  'Sex' 열의 값이 'F' 또는 'M'인 행만 선택하여
```

`df\_test2`라는 새로운 데이터프레임을 만듭니다.

```
`X_cols` = [i for i in df_train2.columns if i != 'Sex']`:
```

- `df\_train2` 데이터프레임의 열 이름 중에서
`Sex` 열을 제외한 모든 열 이름의 리스트를 생성합니다.
- 리스트 컴프리헨션을 사용하여 `df\_train2.columns`의
각 열 이름을 순회하며 `Sex` 열 이름을 제외하고
나머지 열 이름을 리스트에 추가합니다.

전체적인 동작 설명:

1. `df\_train`과 `df\_test` 데이터프레임에서 `Sex` 열이
`F` 또는 `M`인 행만 필터링하여 각각 `df\_train2`와
`df\_test2`라는 새로운 데이터프레임을 생성합니다.
2. `df\_train2` 데이터프레임에서 `Sex` 열을 제외한
모든 열 이름을 `X\_cols` 리스트에 저장합니다.

이 코드의 목적은 주어진 데이터프레임에서 `Sex` 열의 값이
`F` 또는 `M`인 데이터만을 사용하고, 분석에 필요한
열 목록을 준비하는 것입니다. 이 준비 과정은 이후의
데이터 분석이나 모델링 작업에서 사용할 수 있습니다.
'''

```
df_train2 = df_train.loc[df_train['Sex'].isin(['F', 'M'])]
df_test2 = df_test.loc[df_test['Sex'].isin(['F', 'M'])]
```

```
X_cols = [i for i in df_train2.columns if i != 'Sex']
X_cols
```

```
Out[26]: ['Length',
          'Diameter',
          'Height',
          'Whole',
          'Shucked',
          'Viscera',
          'Shell',
          'Rings']
```

```
In [28]: '''
코드는 파이프라인을 사용하여 데이터 전처리와
로지스틱 회귀 모델을 학습시키는
과정을 보여줍니다.
```


1. Pipeline 생성:
`make\_pipeline` 함수를 사용하여 파이프라인 객체를 생성합니다.
2. 데이터 스케일링:
`StandardScaler`를 파이프라인의 첫 번째 단계로 설정합니다. 이는 데이터를 표준화 (평균을 0, 표준편차를 1로 변환)하여 모델의 성능을 향상시키는 역할을 합니다.
3. 로지스틱 회귀 모델:
`LogisticRegression`을 파이프라인의 두 번째 단계로 설정합니다. 이는 로지스틱 회귀 분류 모델을 사용하여 데이터를 학습시킵니다.

1. 파이프라인 생성

- `StandardScaler()`:
입력 데이터의 각 특징을 평균이 0, 표준편차가 1이 되도록 변환합니다.
- `LogisticRegression(random\_state=123, solver='lbfgs')`:
로지스틱 회귀 모델을 설정합니다. `solver='lbfgs'`는 최적화 알고리즘으로 L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno)를 사용합니다.

2. 모델 학습

- `df\_train2[X\_cols]`:
훈련 데이터에서 특징 열(설명 변수)을 선택합니다. `X\_cols`는 특징 열의 리스트입니다.
- `df\_train2['Sex']`:
훈련 데이터에서 목표 변수(종속 변수)인 'Sex' 열을 선택합니다.

`fit` 메서드는 다음을 수행합니다:

- 먼저, `StandardScaler`를 사용하여 입력 데이터 `df\_train2[X\_cols]`를 표준화합니다.
- 그 다음, 표준화된 데이터를 사용하여 `LogisticRegression` 모델을 학습시킵니다.

요약하면, 이 코드는 데이터를 표준화한 후 로지스틱 회귀 모델을 사용하여 성별(Sex)을 예측하는 모델을 학습시키는 과정을 수행합니다.

```
'''
from sklearn.linear_model import LogisticRegression
clf_lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(random_state=123, solver='lbfgs')
)
clf_lr.fit(df_train2[X_cols], df_train2['Sex'])
```

```
Out[28]: Pipeline(memory=None,
                 steps=[('standardscaler',
                        StandardScaler(copy=True, with_mean=True, with_std=True)),
                        ('logisticregression',
                        LogisticRegression(C=1.0, class_weight=None, dual=False,
                                           fit_intercept=True, intercept_scaling=1,
                                           l1_ratio=None, max_iter=100,
                                           multi_class='warn', n_jobs=None,
                                           penalty='l2', random_state=123,
                                           solver='lbfgs', tol=0.0001, verbose=0,
                                           warm_start=False))],
                 verbose=False)
```

```
In [29]: '''
코드는 로지스틱 회귀 모델의 성능을 평가하기 위해
ROC AUC 점수를 계산합니다. ROC AUC 점수는 모델의
예측 성능을 평가하는 데 사용되는 중요한 지표입니다.

1. 모델을 사용하여 예측 확률 계산:
- 훈련 데이터(`df_train2`)와 테스트 데이터(`df_test2`)에
  대해 모델이 각 샘플이 긍정 클래스
  (여기서는 'Sex' 변수의 특정 값,
  예를 들어 'Female')일 확률을 예측합니다.
2. ROC AUC 점수 계산:
- 예측된 확률과 실제 클래스 레이블을 사용하여
  ROC AUC 점수를 계산합니다.

세부 단계별로 설명하면 다음과 같습니다:
### 1. 예측 확률 계산
- `clf_lr.predict_proba(df_train2[X_cols])`는
  훈련 데이터에 대해 각 클래스에 속할 확률을 예측합니다.
- `[:, 1]`은 긍정 클래스(여기서는 'Female')일 확률)만을
```

선택합니다. 예를 들어, ``predict_proba`` 메서드는 클래스 0과 클래스 1에 대한 확률을 반환하는데, 여기서는 클래스 1('Female')에 대한 확률을 사용합니다.

2. ROC AUC 점수 계산

- ``roc_auc_score`` 함수는 실제 클래스 레이블 (``df_train2['Sex']``)과 예측된 확률을 사용하여 ROC AUC 점수를 계산합니다.
- 이 계산은 모델이 얼마나 잘 클래스 'Female'을 예측하는지에 대한 지표를 제공합니다.

3. 테스트 데이터에 대한 ROC AUC 점수 계산

- 위와 동일한 방식으로 테스트 데이터에 대해 ROC AUC 점수를 계산합니다.
- ``df_test2['Sex']``는 테스트 데이터의 실제 클래스 레이블입니다.
- ``clf_lr.predict_proba(df_test2[X_cols])[:, 1]``는 테스트 데이터에 대해 예측된 긍정 클래스 확률입니다.

최종 결과

- 훈련 데이터와 테스트 데이터 각각에 대해 계산된 ROC AUC 점수를 튜플 형태로 반환합니다.

이 코드는 모델의 성능을 두 가지 데이터셋 (훈련 데이터와 테스트 데이터)에서 평가하고, 각 데이터셋에 대해 ROC AUC 점수를 계산하여 모델의 예측 성능을 확인합니다.

```
from sklearn.metrics import roc_auc_score

(
    roc_auc_score(df_train2['Sex'], \
                  clf_lr.predict_proba(df_train2[X_cols])[:, 1]),
    roc_auc_score(df_test2['Sex'], \
                  clf_lr.predict_proba(df_test2[X_cols])[:, 1])
)
```

Out[29]: (0.5866224253815038, 0.558851397589093)

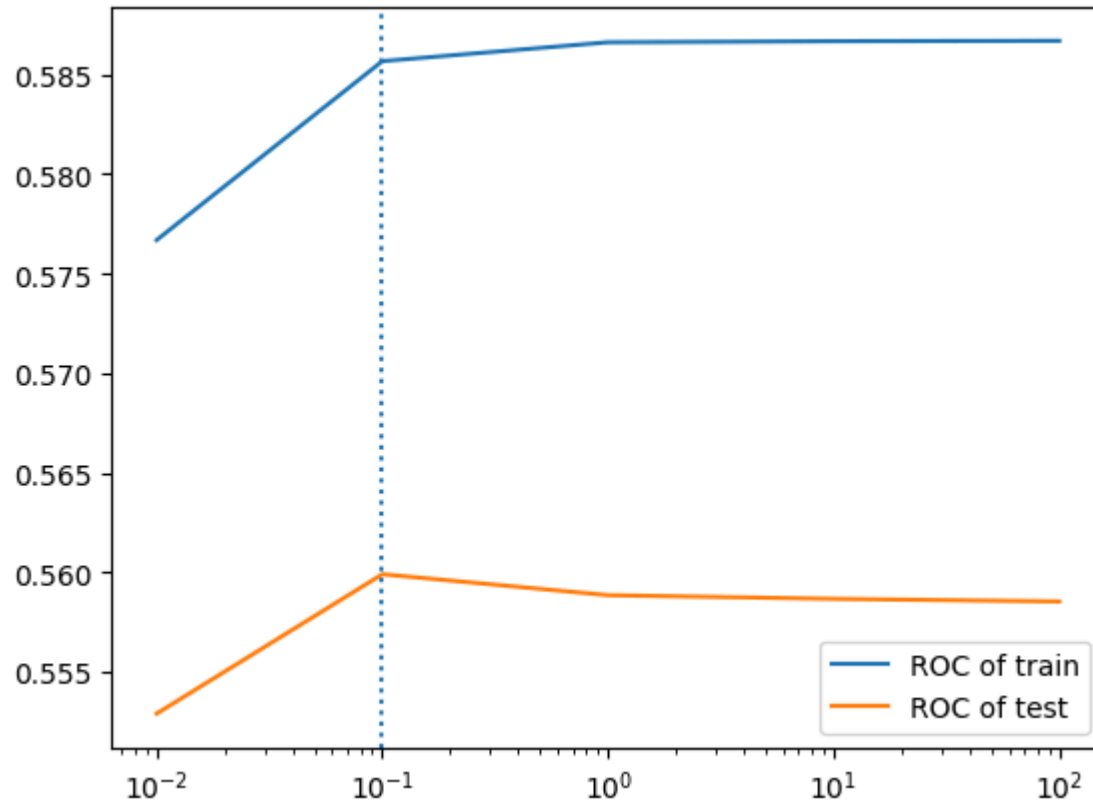
C에 따라서 df_train2, df_test2의 성능의 변화 정도를 확인해 봅니다.

```
C = [1e-2, 1e-1, 1, 1e1, 1e2]
```

```
In [30]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
params = [1e-2, 1e-1, 1, 1e1, 1e2]

roc_train, roc_test = list(), list()
for param in params:
    clf_lr = make_pipeline(
        StandardScaler(),
        LogisticRegression(C=param, random_state=123,
                           solver='lbfgs')
    )
    clf_lr.fit(df_train2[X_cols], df_train2['Sex'])
    roc_train.append(roc_auc_score(df_train2['Sex'],
                                   clf_lr.predict_proba(df_train2[X_cols])[:, 1]))
    roc_test.append(roc_auc_score(df_test2['Sex'],
                                   clf_lr.predict_proba(df_test2[X_cols])[:, 1]))
```

```
In [31]: plt.gca().set(xscale='log')
sns.lineplot(x = params, y = roc_train, label='ROC of train')
sns.lineplot(x = params, y = roc_test, label='ROC of test')
plt.axvline(x = params[np.argmax(roc_test)], linestyle='dotted')
plt.legend()
plt.show()
```



이진 분류일 때, `sklearn.linear_model.LogisticRegression`의 `decision_function`은 Logit을 반환합니다.

```
In [32]: '''
`decision_function()` 함수는 주로 Scikit-learn
라이브러리에서 사용되며, 분류 모델이 각 클래스에 대해
산출하는 "결정 점수"를 반환합니다.
이 점수는 클래스에 대한 신뢰도를 나타내며,
예측된 클래스의 선택에 영향을 미칩니다.

### 주요 모델별 `decision_function()` 설명

1. SVM (Support Vector Machines):
```

- 설명: `SVC` 또는 `LinearSVC` 클래스의 `decision\_function()`은 각 샘플에 대해 초평면에서의 거리를 나타내는 값을 반환합니다.
- 인자:
 - `X`: 입력 데이터. 형태는 `(n\_samples, n\_features)`.
- 리턴값:
 - 이진 분류의 경우: 배열 형태의 결정 점수. 각 샘플에 대해 하나의 값이 반환됩니다. 양수 값은 양의 클래스로, 음수 값은 음의 클래스로 분류됩니다.
 - 다중 클래스 분류의 경우: 배열 형태의 결정 점수. 각 샘플에 대해 `n\_classes`개의 점수가 반환되며, 각 클래스에 대한 결정 점수입니다.
- 예시:


```
from sklearn.svm import SVC
model = SVC()
model.fit(X_train, y_train)
decision_scores = model.decision_function(X_test)
```

2. Logistic Regression:

- 설명: `LogisticRegression` 클래스의 `decision\_function()`은 각 샘플에 대해 선형 모델의 결함을 나타내는 점수를 반환합니다.
- 인자:
 - `X`: 입력 데이터. 형태는 `(n\_samples, n\_features)`.
- 리턴값:
 - 이진 분류의 경우: 배열 형태의 결정 점수. 각 샘플에 대해 하나의 값이 반환됩니다.
 - 다중 클래스 분류의 경우: 배열 형태의 결정 점수. 각 샘플에 대해 `n\_classes`개의 점수가 반환됩니다.
- 예시:


```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
decision_scores = model.decision_function(X_test)
```

3. SGD Classifier (Stochastic Gradient Descent):

- 설명: `SGDClassifier` 클래스의 `decision\_function()`은

SVM이나 Logistic Regression과 유사한 방식으로
각 샘플에 대해 결정 점수를 반환합니다.

- 인자:
 - `X`: 입력 데이터. 형태는 `(n\_samples, n\_features)`.
- 리턴값:
 - 이진 분류의 경우:
 - 배열 형태의 결정 점수.
 - 각 샘플에 대해 하나의 값이 반환됩니다.
 - 다중 클래스 분류의 경우:
 - 배열 형태의 결정 점수. 각 샘플에 대해
`n\_classes`개의 점수가 반환됩니다.

```
- 예시:
from sklearn.linear_model import SGDClassifier
model = SGDClassifier()
model.fit(X_train, y_train)
decision_scores = model.decision_function(X_test)
```

결정 점수의 사용

결정 점수는 다음과 같은 목적으로 사용할 수 있습니다:

- 예측 클래스 결정:
 - 결정 점수는 가장 높은 점수를 가진 클래스를
예측하기 위해 사용됩니다.
- 모델의 신뢰도 평가:
 - 결정 점수의 절대값을 통해 모델의 예측에 대한
신뢰도를 평가할 수 있습니다.
- ROC 커브 및 AUC 계산:
 - 이진 분류 문제에서 결정 점수를 사용하여
ROC 커브를 그리고, AUC (Area Under the Curve)를
계산할 수 있습니다.

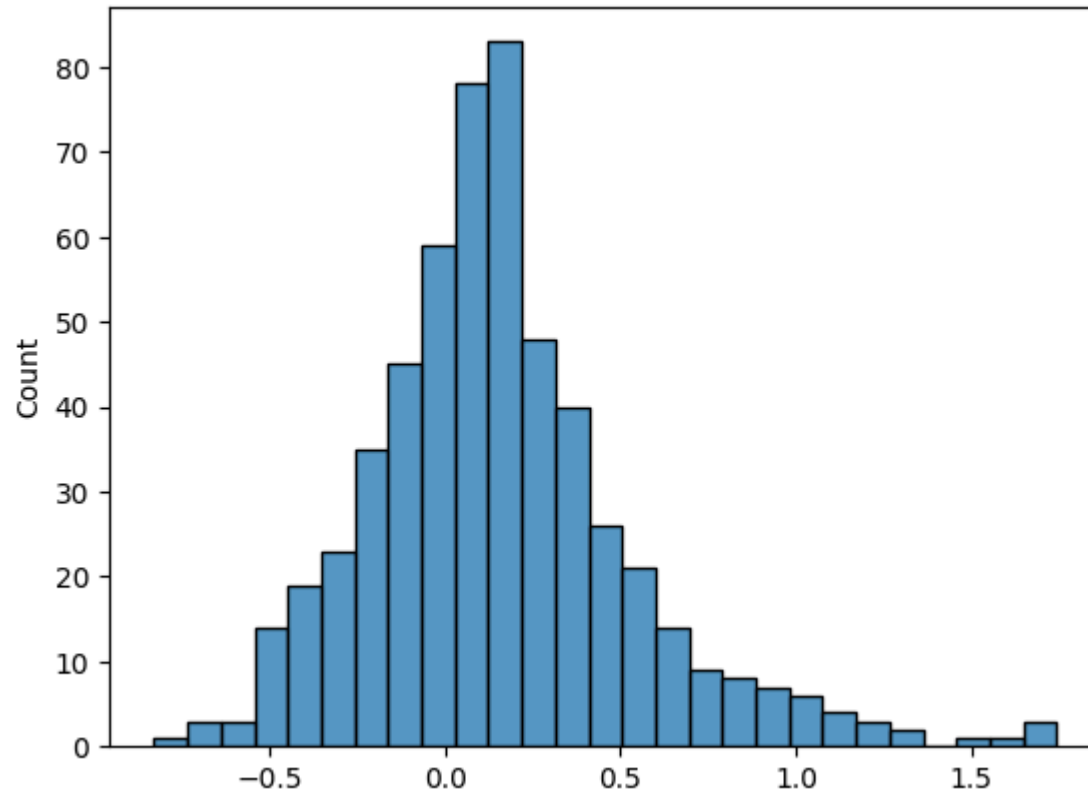
```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt
# 이진 분류 문제에서 decision_function() 결과를
# 사용하여 ROC 커브 그리기
fpr, tpr, thresholds = roc_curve(y_test, decision_scores)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2,
         label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2,
```

```
        linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()

위의 예시에서 `decision_function()`의 결과를 이용하여
ROC 커브를 그리고, AUC 값을 계산할 수 있습니다.
이를 통해 모델의 성능을 시각적으로 평가할 수 있습니다.
'''

sns.histplot(
    clf_lr.decision_function(df_test2[X_cols])
)
plt.show()
```

In [2]: `Image('163.png')`

Out[2]: Logit을 Logistic 함수에 전달하면 Positive Class일 확률이 나오게 됩니다.

$$\hat{y} = \frac{e^x}{1+e^x} = \frac{1}{1+e^{-x}}$$

In [33]: `from scipy.special import expit`

```
#expit(clf_lr.decision_function(df_test2[X_cols]))
1 / (1 + np.exp(-clf_lr.decision_function(df_test2[X_cols]]))[:5]
```

Out[33]: `array([0.73800659, 0.62081834, 0.45495903, 0.46829104, 0.53452528])`

```
In [34]: '''
코드는 로지스틱 회귀 모델 `clf_lr`을 사용하여
테스트 데이터 `df_test2`의 첫 5개의 샘플에 대해
긍정 클래스(예: 'Female')의 예측 확률을 추출하는
작업을 수행합니다.

### 1. 예측 확률 계산
- `clf_lr`는 이미 학습된 로지스틱 회귀 모델입니다.
- `df_test2[X_cols]`는 테스트 데이터에서 모델에
  사용할 특징 열(설명 변수)입니다.
- `predict_proba` 메서드는 각 샘플에 대해
  각 클래스(여기서는 'Male'과 'Female')에 속할
  확률을 반환합니다. 반환된 결과는 2차원 배열로,
  각 행은 샘플을 나타내고 각 열은 클래스에 대한
  확률을 나타냅니다.

### 2. 긍정 클래스의 확률 추출
- `[:, 1]`은 모든 샘플에 대해 두 번째 열
  (긍정 클래스, 여기서는 'Female')의 확률을 선택합니다.
  예를 들어, `predict_proba`가 반환하는 배열의 형식은 다음과 같습니다:
  [[0.3, 0.7], # 첫 번째 샘플의 확률: [클래스 0('Male'), 클래스 1('Female')],
  [0.8, 0.2], # 두 번째 샘플의 확률: [클래스 0('Male'), 클래스 1('Female')],
  ...]
  여기서 두 번째 열(즉, `[:, 1]`)을 선택하면
  클래스 1('Female')의 확률을 얻게 됩니다.

### 3. 첫 5개 샘플의 확률 추출
- `[:5, 1]`은 첫 5개의 샘플에 대해 긍정 클래스
  (클래스 1, 'Female')의 확률을 선택합니다.
  결과는 1차원 배열로, 각 값은 첫 5개 샘플에 대해
  'Female'일 확률을 나타냅니다.

### 요약
이 코드는 테스트 데이터 `df_test2`에서 첫 5개의
샘플에 대해 로지스틱 회귀 모델 `clf_lr`이 예측한
'Female' 클래스에 대한 확률을 추출합니다.
예를 들어, 반환되는 값이 `[0.7, 0.2, 0.6, 0.8, 0.5]`라면,
이는 첫 번째 샘플이 70% 확률로 'Female',
두 번째 샘플이 20% 확률로 'Female', ... ,
다섯 번째 샘플이 50% 확률로 'Female'이라는 것을 의미합니다.
'''
```

```
'''
clf_lr.predict_proba(df_test2[X_cols][:5, 1])
```

Out[34]: array([0.73800659, 0.62081834, 0.45495903, 0.46829104, 0.53452528])

In [3]: Image('164.png')

Out[3]: Positive Class일 확률을 가지고 Logit을 만들어 봅니다.

$$\text{logit}(\hat{y}) = \log\left(\frac{\hat{y}}{1-\hat{y}}\right)$$

```
In [35]: '''
코드는 테스트 데이터 `df_test2`의 첫 5개 샘플에
대해 로지스틱 회귀 모델 `clf_lr`의 로그 오즈
(log odds)를 계산하는 작업을 수행합니다.

### 1. 예측 확률 계산
- `clf_lr`는 학습된 로지스틱 회귀 모델입니다.
- `df_test2[X_cols]`는 테스트 데이터에서
  모델에 사용할 특징 열(설명 변수)입니다.
- `predict_proba` 메서드는 각 샘플에 대해
  각 클래스에 속할 확률을 반환합니다.
  결과는 2차원 배열로, 각 행은 샘플을 나타내고
  각 열은 해당 클래스에 대한 확률을 나타냅니다.

### 2. 긍정 클래스와 부정 클래스의 확률 추출
- `[ , 1]`은 각 샘플에 대해 긍정 클래스
  (예: 'Female')의 확률을 선택합니다.
- `[ , 0]`은 각 샘플에 대해 부정 클래스
  (예: 'Male')의 확률을 선택합니다.

### 3. 오즈 계산
- 긍정 클래스의 확률을 부정 클래스의 확률로
  나누어 오즈(odds)를 계산합니다.
  오즈는 특정 사건(여기서는 'Female')이 발생할
  확률을 그 사건이 발생하지 않을 확률로 나눈 값입니다.
- 예를 들어, 한 샘플의 확률이 0.7 (긍정 클래스)와
```

0.3 (부정 클래스)라면,
오즈는 $\left(\frac{0.7}{0.3} \right) = 2.33$ 이 됩니다.

4. 로그 오즈 계산

- 오즈의 자연 로그(natural logarithm)를 계산합니다.
로그 오즈는 로지스틱 회귀 모델의 예측 결과를 해석하는 데 자주 사용됩니다.
- 자연 로그를 취하면 오즈가 지수 함수로 변환된 것이기 때문에 로그 오즈는 선형 모델의 출력과 직접적으로 관련이 있습니다.

5. 첫 5개 샘플의 로그 오즈 추출

- 첫 5개 샘플에 대한 로그 오즈 값을 선택합니다.
결과는 1차원 배열로, 각 값은 첫 5개 샘플에 대한 로그 오즈를 나타냅니다.

요약

이 코드는 테스트 데이터 `df\_test2`의 첫 5개 샘플에 대해 로지스틱 회귀 모델 `clf\_lr`이 예측한 로그 오즈를 계산합니다. 각 샘플에 대해 로그 오즈는 해당 샘플이 긍정 클래스(예: 'Female')일 로그 오즈 비율을 나타냅니다.
예를 들어, 결과가 `[0.85, -0.45, 1.2, -1.0, 0.3]`라면, 이는 첫 번째 샘플의 로그 오즈가 0.85, 두 번째 샘플의 로그 오즈가 -0.45, ... , 다섯 번째 샘플의 로그 오즈가 0.3이라는 것을 의미합니다.
...

```
from scipy.special import logit

#logit(clf_lr.predict_proba(df_test2[X_cols]))[:, 1])
np.log(clf_lr.predict_proba(df_test2[X_cols]))[:, 1] \
    / clf_lr.predict_proba(df_test2[X_cols]))[:, 0]))[:5]
```

Out[35]: array([1.03563338, 0.49302312, -0.18065359, -0.12700628, 0.13832123])

```
In [3]: import pandas as pd
        from sklearn.linear_model import LogisticRegression
```

예시 데이터 생성

```
data = {'feature1': [0.5, 0.6, 0.8, 0.9, 1.0],
        'feature2': [1.0, 0.9, 0.8, 0.7, 0.6],
        'feature3': [0.3, 0.2, 0.1, 0.4, 0.5]}
df_test2 = pd.DataFrame(data)
X_cols = ['feature1', 'feature2', 'feature3']

# 예시 모델 생성 및 학습
clf_lr = LogisticRegression()
X_train = [[0.5, 1.0, 0.3], [0.6, 0.9, 0.2], [0.8, 0.8, 0.1], [0.9, 0.7, 0.4], [1.0, 0.6, 0.5]]
y_train = [0, 0, 1, 1, 1]
clf_lr.fit(X_train, y_train)

# decision_function 결과 확인
decision_scores = clf_lr.decision_function(df_test2[X_cols])
print(decision_scores[:5])
```

```
[0.24865509 0.30725587 0.40520825 0.49876356 0.5748416 ]
```

In [36]:

```
'''
코드는 `clf_lr`라는 분류 모델의 `decision_function()`
메서드를 호출하여, 테스트 데이터 `df_test2`의
일부 컬럼(`X_cols`)에 대한 결정 점수를 계산하고,
그 결과의 첫 5개 값을 반환합니다.

### 각 부분 설명
1. `df_test2`:
   - 이는 테스트 데이터셋을 담고 있는 데이터프레임입니다.
   `pandas.DataFrame` 객체로, 모델이 예측을 수행할 때
   사용할 데이터가 들어 있습니다.
2. `X_cols`:
   - 이는 입력 데이터의 컬럼 이름을 담고 있는 리스트입니다.
   모델이 예측할 때 사용할 특성(피처)들이 포함되어 있습니다.
3. `df_test2[X_cols]`:
   - 이는 테스트 데이터셋의 일부 컬럼을 선택한 것입니다.
   `X_cols`에 포함된 컬럼들만 선택하여
   새로운 데이터프레임을 생성합니다.
4. `clf_lr.decision_function(df_test2[X_cols])`:
   - `decision_function()` 메서드는 입력 데이터에 대해
   각 클래스에 대한 결정 점수를 계산합니다.
   이 점수는 모델이 각 클래스에 속할 확률을
   계산하기 전에 내부적으로 사용하는 값입니다.
'''
```

```

- 로지스틱 회귀 모델의 경우,
이 점수는 각 샘플에 대해 선형 모델의 결합 결과로서,
이진 분류일 때는 샘플이 양성 클래스에 속할
가능성을 나타내며, 다중 클래스 분류일 때는
각 클래스에 대한 점수 배열을 반환합니다.
5. `[:5]`:
- 이는 슬라이싱을 통해 결정 점수 배열의
첫 5개 값을 선택합니다. 이를 통해 전체 결정 점수 중
처음 5개의 샘플에 대한 점수만을 확인할 수 있습니다.
### 결론
해당 코드는 테스트 데이터의 일부 특성들에 대해
학습된 분류 모델이 산출한 결정 점수를 계산하고,
그 결과의 첫 5개 값을 반환합니다.
이는 모델이 새로운 데이터에 대해 어떻게 결정하는지
이해하는 데 도움을 줄 수 있으며,
특히 모델의 예측 신뢰도를 평가하거나
결과를 분석할 때 유용합니다.
'''

clf_lr.decision_function(df_test2[X_cols])[:5]

```

```
Out[36]: array([ 1.03563338,  0.49302312, -0.18065359, -0.12700628,  0.13832123])
```

[Ex.8]

성별(Sex)을 맞추는 로지스틱 회귀 모델을 만들어 df_train로 학습을 시키고,

이 때, 입력 변수들을 표준화합니다.

multi_class가 'ovr'(One vs Rest) 일때와 multi_class가 'multinomial'의 작업의 의미를 살펴봅니다.

```

In [37]: '''
1. `make_pipeline`:
- `make_pipeline`은 Scikit-learn에서 제공하는 함수로,
여러 개의 변환기와 추정기를 순차적으로 적용하는
파이프라인을 생성합니다. 이를 통해 데이터 전처리와
모델 학습을 하나의 객체로 관리할 수 있습니다.

```

```

2. `StandardScaler`:
- `StandardScaler`는 데이터의 평균을 0, 표준편차를 1로
  맞추는 정규화 작업을 수행합니다.
  이는 모델 학습 전에 데이터의 스케일을 조정하여
  성능을 향상시키기 위해 사용됩니다.
3. `LogisticRegression`:
- `LogisticRegression`은 로지스틱 회귀 모델을 생성하는
  클래스입니다. 이 모델은 분류 문제를 해결하는 데
  사용됩니다.
- `solver='lbfgs'`는 최적화 알고리즘으로 L-BFGS
  (Limited-memory Broyden-Fletcher-Goldfarb-Shanno)를
  사용합니다. 이는 다차원 문제에 적합한 알고리즘입니다.
- `multi_class='ovr'`는 다중 클래스 분류에서
  '일대다 (one-vs-rest)' 방식을 사용하여 각 클래스를
  독립적으로 이진 분류하는 방식입니다.
4. `clf_lr = make_pipeline(...)`:
- `StandardScaler`와 `LogisticRegression`을 순차적으로
  적용하는 파이프라인 객체를 생성합니다. 이 파이프라인은
  먼저 데이터를 정규화하고, 그 후 로지스틱 회귀 모델을
  적용하는 작업을 수행합니다.
5. `clf_lr.fit(df_train[X_cols], df_train['Sex']):`
- `fit` 메서드는 파이프라인을 통해 데이터를 학습하는 과정입니다.
  이 메서드는 파이프라인에 포함된 모든 변환기를
  순차적으로 적용하여 데이터를 변환한 후,
  최종 모델을 학습시킵니다.
- `df_train`은 학습 데이터셋을 담고 있는 데이터프레임입니다.
- `X_cols`는 입력 데이터의 컬럼 이름을 담고 있는 리스트입니다.
- `df_train['Sex']`는 타겟 변수로,
  예측하려는 클래스 레이블입니다.
  여기서는 'Sex' 컬럼을 타겟 변수로 사용하고 있습니다.

```

결론

이 코드의 전체 동작 과정은 다음과 같습니다:

1. 데이터 전처리와 모델 설정: `StandardScaler`로 데이터를 정규화하고, `LogisticRegression` 모델을 사용하여 분류 문제를 해결하는 파이프라인을 생성합니다.
2. 모델 학습: 학습 데이터 (`df\_train[X\_cols]`와 `df\_train['Sex']`)를 사용하여 파이프라인을 학습시킵니다. 이 과정에서 먼저 데이터를 정규화한 후, 정규화된 데이터를 사용하여 로지스틱 회귀 모델을 학습합니다.

이 파이프라인을 사용하면 데이터 전처리와 모델 학습을 하나의 객체로 묶어서 관리할 수 있으며, 학습된 모델을 사용하여 새로운 데이터에 대해 예측을 수행할 수 있습니다.

```
'''
clf_lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(random_state=123,
                       solver='lbfgs', multi_class='ovr')
)
clf_lr.fit(df_train[X_cols], df_train['Sex'])
'''
```

```
Out[37]: Pipeline(memory=None,
                 steps=[('standardscaler',
                        StandardScaler(copy=True, with_mean=True, with_std=True)),
                        ('logisticregression',
                        LogisticRegression(C=1.0, class_weight=None, dual=False,
                                           fit_intercept=True, intercept_scaling=1,
                                           l1_ratio=None, max_iter=100,
                                           multi_class='ovr', n_jobs=None,
                                           penalty='l2', random_state=123,
                                           solver='lbfgs', tol=0.0001, verbose=0,
                                           warm_start=False))],
                 verbose=False)
```

```
In [4]: '''
예시는 테스트 데이터의 스케일을 학습 데이터와
동일하게 맞추는 과정을 보여줍니다.
이는 학습 데이터와 테스트 데이터가 동일한 스케일을
가지도록 보장하여 모델의 성능을 일관되게
유지할 수 있도록 합니다. - 아래와 관련된 예제임
'''

import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 예시 데이터 생성
data_train = {'feature1': [0.5, 0.6, 0.8, 0.9, 1.0],
              'feature2': [1.0, 0.9, 0.8, 0.7, 0.6],
```



```

        'feature3': [0.3, 0.2, 0.1, 0.4, 0.5],
        'Sex': [0, 0, 1, 1, 1]}
df_train = pd.DataFrame(data_train)
data_test = {'feature1': [0.4, 0.7, 0.8, 0.9, 1.1],
             'feature2': [1.1, 0.8, 0.7, 0.6, 0.5],
             'feature3': [0.2, 0.3, 0.4, 0.5, 0.6]}
df_test = pd.DataFrame(data_test)
X_cols = ['feature1', 'feature2', 'feature3']

# 파이프라인 생성 및 학습
clf_lr = make_pipeline(StandardScaler(),
                       LogisticRegression(random_state=123,
                                           solver='lbfgs',
                                           multi_class='ovr'))

clf_lr.fit(df_train[X_cols], df_train['Sex'])

# 테스트 데이터 변환
X_test_std = clf_lr[0].transform(df_test[X_cols])
print(X_test_std)

```

```

[[-1.94098992  2.12132034 -0.70710678]
 [-0.32349832  0.          0.          ]
 [ 0.21566555 -0.70710678  0.70710678]
 [ 0.75482941 -1.41421356  1.41421356]
 [ 1.83315714 -2.12132034  2.12132034]]

```

In [38]:

```

'''
해당 코드는 Scikit-learn 파이프라인에서 전처리
단계만을 사용하여 테스트 데이터를 변환하는
작업을 수행합니다.

1. `clf_lr`:
- `clf_lr`는 이전 코드에서 정의된 파이프라인 객체입니다.
이 파이프라인은 `StandardScaler`와
`LogisticRegression`으로 구성되어 있습니다.
2. `clf_lr[0]`:
- 파이프라인 객체는 리스트처럼 인덱싱을 통해
각 단계에 접근할 수 있습니다. `clf_lr[0]`는 파이프라인의
첫 번째 단계인 `StandardScaler` 객체를 의미합니다.
3. `df_test[X_cols]`:
- `df_test`는 테스트 데이터셋을 담고 있는
'''

```

```

    데이터프레임입니다.
    - `X_cols`는 입력 데이터의 컬럼 이름을 담고 있는 리스트입니다.
      이 컬럼들만 선택하여 새로운 데이터프레임을 생성합니다.
4. `clf_lr[0].transform(df_test[X_cols])`:
    - `transform` 메서드는 `StandardScaler` 객체에 의해
      데이터의 스케일을 변환합니다. 이 메서드는 학습 데이터의
      평균과 표준편차를 사용하여 테스트 데이터를 정규화합니다.

### 결론
이 코드의 전체 동작 과정은 다음과 같습니다:
1. 테스트 데이터 준비:
    테스트 데이터셋(`df_test`)에서 예측에 사용할
    특성들(`X_cols`)을 선택합니다.
2. 데이터 변환: 학습 데이터로 계산된 평균과 표준편차를
    사용하여 테스트 데이터를 정규화합니다. 이를 통해
    테스트 데이터의 스케일을 학습 데이터와 동일하게 맞춥니다.
...
X_test_std = clf_lr[0].transform(df_test[X_cols])

```

```

In [5]: '''
예시에서는 StandardScaler로 데이터 전처리를 수행하고,
LogisticRegression 모델을 학습시킵니다.
그런 다음 학습된 모델의 계수(coef_)와
절편(intercept_) 값을 추출하여 출력합니다.
- 아래와 관련된 예제임
'''

import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 예시 데이터 생성
data_train = {'feature1': [0.5, 0.6, 0.8, 0.9, 1.0],
               'feature2': [1.0, 0.9, 0.8, 0.7, 0.6],
               'feature3': [0.3, 0.2, 0.1, 0.4, 0.5],
               'Sex': [0, 0, 1, 1, 1]}
df_train = pd.DataFrame(data_train)
X_cols = ['feature1', 'feature2', 'feature3']

```

```
# 파이프라인 생성 및 학습
clf_lr = make_pipeline(StandardScaler(),
                        LogisticRegression(random_state=123,
                                           solver='lbfgs',
                                           multi_class='ovr'))

clf_lr.fit(df_train[X_cols], df_train['Sex'])

# 학습된 로지스틱 회귀 모델의 계수와 절편 값 확인
coefficients = clf_lr[1].coef_
intercepts = clf_lr[1].intercept_

print("Coefficients:", coefficients)
print("Intercepts:", intercepts)
```

```
Coefficients: [[ 0.82064742 -0.70031862  0.0042332 ]]
Intercepts: [0.60586117]
```

In [39]:

```
'''
해당 코드는 Scikit-learn 파이프라인에서 학습된
로지스틱 회귀 모델의 계수(coefficient)와
절편(intercept)을 가져오는 작업을 수행합니다.

1. `clf_lr`:
   - `clf_lr`는 이전 코드에서 정의된 파이프라인
     객체입니다. 이 파이프라인은 `StandardScaler`와
     `LogisticRegression`으로 구성되어 있습니다.
2. `clf_lr[1]`:
   - 파이프라인 객체는 리스트처럼 인덱싱을 통해
     각 단계에 접근할 수 있습니다. `clf_lr[1]`은
     파이프라인의 두 번째 단계인
     `LogisticRegression` 객체를 의미합니다.
3. `logistic_regression_model.coef_`:
   - `coef_` 속성은 학습된 로지스틱 회귀 모델의
     계수(혹은 가중치)들을 반환합니다.
     이는 각 특성(feature)이 예측에 얼마나
     영향을 미치는지를 나타냅니다.
   - 다중 클래스 분류의 경우, 각 클래스에 대한
     계수를 행렬 형태로 반환합니다.
4. `logistic_regression_model.intercept_`:
   - `intercept_` 속성은 학습된 로지스틱 회귀 모델의
     절편(혹은 바이어스) 값을 반환합니다.
'''
```

이는 모델의 예측을 보정하기 위해 사용됩니다.
 - 다중 클래스 분류의 경우, 각 클래스에 대한
 절편 값을 배열 형태로 반환합니다.

결론

이 코드는 학습된 로지스틱 회귀 모델에서 각 특성의 계수와
 절편 값을 추출하여 반환합니다. 이는 모델이 각 특성을
 어떻게 가중치 두고 평가하는지,
 예측을 어떻게 조정하는지에 대한 정보를 제공합니다.
 ...

```
clf_lr[1].coef_, clf_lr[1].intercept_
```

```
Out[39]: (array([[ -0.01289247,  1.0203668 ,  0.23410588,  0.15956636, -0.7117244 ,
                   0.39013469, -0.23430952,  0.1573919 ],
                  [ 1.8503066 , -0.7713847 , -0.2266418 , -1.74907003,  0.15134294,
                  -1.48124379,  0.304532 , -0.66902424],
                  [-0.20892971,  0.3043963 ,  0.03182928, -0.1664884 ,  0.58584343,
                   0.083093 , -0.21802962,  0.26162426]]),
          array([-0.93339435, -1.59994996, -0.6009166 ]))
```

multi_class를 ovr로 할 경우에는

'F', 'I', 'M' 클래스를 차례대로 Positive Class로 만들어

로지스틱 회귀모델 이진 분류 3개를 만듭니다.

그리고 Positive 확률(또는 Logit)이 가장 큰 클래스로 예측합니다.

```
In [40]: '''
코드는 세 가지 다른 성별 카테고리('F', 'I', 'M')에
대해 개별적으로 로지스틱 회귀 분류 모델을 학습하고,
각 모델의 계수와 절편을 출력하는 작업을 수행합니다.

### 각 부분 설명
1. `clf_lr_sub`:
- `clf_lr_sub`는 각 성별 카테고리에 대한
로지스틱 회귀 분류 모델을 포함하는 파이프라인입니다.
각 모델은 `StandardScaler`로 전처리된 후,
`LogisticRegression`을 사용하여 학습됩니다.
```

```

2. `for i in ['F', 'I', 'M']:`:
    - 이는 성별 카테고리('F', 'I', 'M')를
      반복하는 루프입니다. 각 성별 카테고리에 대해
      모델을 학습하고 출력합니다.
3. `clf_lr_sub.fit(df_train[X_cols],
                  df_train['Sex'] == i)`:
    - 각 성별 카테고리에 대해 해당하는 데이터를 학습합니다.
      여기서는 성별이 해당 카테고리에 속하는지를
      이진 분류로 수행합니다.
    - 예를 들어, 'F'에 대한 모델은 여성인지
      아닌지를 분류하는 모델이 됩니다.
4. `print(i, clf_lr_sub[1].coef_,
          clf_lr_sub[1].intercept_)`:
    - 학습된 모델의 계수와 절편을 출력합니다.
      각 성별 카테고리에 대해 모델을 따로 학습했으므로,
      각 성별 카테고리에 대한 계수와 절편이 출력됩니다.
    - `clf_lr_sub[1]`은 파이프라인의 두 번째 단계인
      `LogisticRegression` 객체를 의미합니다.

```

결론

이 코드는 각 성별 카테고리('F', 'I', 'M')에 대해 별도의 이진 분류 모델을 학습하고, 각 모델의 계수와 절편을 출력합니다. 이를 통해 성별에 따른 데이터 패턴을 모델링하고, 각 성별에 대한 예측을 수행할 수 있습니다.

```
'''
```

```
'''
```

`clf\_lr\_sub.fit(df\_train[X\_cols], df\_train['Sex'] == i)`
부분을 자세히 설명하겠습니다.

인자들:

```

1. `df_train[X_cols]`:
    - 이는 학습 데이터셋에서 예측에 사용할 특성들을
      선택한 것입니다. `df_train`은 학습 데이터셋을
      담고 있는 데이터프레임이며, `[X_cols]`는
      선택한 특성들의 리스트입니다.
    - 학습 데이터셋의 특성들만을 사용하여
      모델을 학습시키기 위해 사용됩니다.

```

```

2. `df_train['Sex'] == i`:
- 이는 성별이 특정 카테고리에 속하는지를 나타내는
  불리언(Boolean) 시리즈를 생성합니다.
  여기서 `i`는 반복문을 통해
  'F', 'I', 'M' 중 하나의 값을 가집니다.
- 불리언 시리즈는 해당 카테고리에 속하는 경우
  `True`, 그렇지 않은 경우 `False`를 가집니다.
- 예를 들어, `df_train['Sex'] == 'F'`는 성별이
  여성인지 아닌지를 나타내는
  불리언 시리즈를 생성합니다.

### 동작:
- `LogisticRegression` 모델은 이진 분류를 수행하므로,
  타겟 변수는 이진 레이블이어야 합니다.
  따라서 성별 카테고리를 하나의 카테고리로
  선택하여 모델을 학습합니다.
- `df_train[X_cols]`는 학습 데이터셋에서 선택된
  특성들을 가지고 있습니다.
  이는 모델이 예측에 사용할 입력 변수들입니다.
- `df_train['Sex'] == i`는 해당 성별 카테고리에
  속하는지를 나타내는 불리언 시리즈입니다.
  이를 통해 해당 카테고리에 속하는 샘플들을 얻을 수 있습니다.
- 모델은 이렇게 생성된 입력 변수와 해당 성별 카테고리에
  속하는 샘플들을 사용하여 학습됩니다.
  여기서 `True`는 1로, `False`는 0으로
  인코딩되어 이진 분류를 위한 타겟 변수로 사용됩니다.
...

clf_lr_sub = make_pipeline(
    StandardScaler(),
    LogisticRegression(random_state=123,
                       solver='lbfgs', multi_class='ovr')
)
for i in ['F', 'I', 'M']:
    clf_lr_sub.fit(df_train[X_cols], df_train['Sex'] == i)
    print(i, clf_lr_sub[1].coef_, clf_lr_sub[1].intercept_)

```

```
F [[-0.01289247  1.0203668   0.23410588  0.15956636 -0.7117244   0.39013469
    -0.23430952  0.1573919   ] [-0.93339435]
I [[ 1.8503066  -0.7713847  -0.2266418  -1.74907003  0.15134294 -1.48124379
    0.304532   -0.66902424] [-1.59994996]
M [[-0.20892971  0.3043963   0.03182928 -0.1664884   0.58584343  0.083093
    -0.21802962  0.26162426] [-0.6009166]
```

```
In [6]: Image('165.png')
```

```
Out[6]: multi_class를 multinomial일 경우에는 Softmax로
```

손실(크로스 엔트로피)을 계산하는 로지스틱 회귀 모델을 만듭니다.

$$\text{softmax}(p) = \frac{e^{p_i}}{\sum_{i=1}^c e^{p_i}}, c: \text{클래스의 수}$$

$$L_{\text{Logistic}}(y, \hat{y}) = - \sum_{i=1}^c y_i \log(\hat{y}_i), c: \text{클래스의 수}, y_i: y \text{ 벡터가 } i \text{ 클래스 여부를 나타내는 벡터}, \hat{y}_i: i \text{ 클래스 가능성 나타내는 예측}$$

```
In [41]: '''
이 코드는 다중 클래스 분류를 위한 로지스틱 회귀
모델을 학습하는 작업을 수행합니다.

### 각 부분 설명
1. `clf_lr`:
    - `clf_lr`은 Scikit-learn의 파이프라인 객체입니다.
      이 파이프라인은 데이터 전처리 단계와 모델 학습
      단계를 한 번에 관리할 수 있도록 합니다.
    - 이 파이프라인은 `StandardScaler`로 데이터를 표준화하고,
      `LogisticRegression`으로 로지스틱 회귀 모델을 생성합니다.
    - `StandardScaler`는 평균이 0이고 표준편차가 1이
      되도록 데이터를 변환합니다.
    - `LogisticRegression`에서 `multi_class` 매개변수를
      'multinomial'로 설정하여 다중 클래스 분류를
      수행하도록 지정했습니다.
2. `clf_lr.fit(df_train[X_cols], df_train['Sex']):`
    - `fit` 메서드는 파이프라인을 통해 데이터를 학습하는
      작업을 수행합니다. 이 메서드는 파이프라인에 포함된
      모든 변환기를 순차적으로 적용한 후, 최종 모델을 학습합니다.
```

- `df_train[X_cols]`는 학습 데이터셋에서 예측에 사용할 특성들을 선택한 것입니다.
`df_train`은 학습 데이터셋을 담고 있는 데이터프레임이며,
`[X_cols]`는 선택한 특성들의 리스트입니다.
- `df_train['Sex']`는 타겟 변수로, 예측하려는 클래스 레이블입니다.

결론

이 코드는 다중 클래스 분류를 위한 로지스틱 회귀 모델을 학습합니다. 이를 위해 데이터를 표준화하고(`StandardScaler`), 로지스틱 회귀 모델을 생성하며 다중 클래스 분류를 수행할 수 있도록 설정합니다(`LogisticRegression`의 `multi_class='multinomial'` 옵션). 그런 다음 학습 데이터로 모델을 학습시킵니다.

```
clf_lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(random_state=123,
                       solver='lbfgs',
                       multi_class='multinomial')
)
clf_lr.fit(df_train[X_cols], df_train['Sex'])
```

```
Out[41]: Pipeline(memory=None,
                 steps=[('standardscaler',
                        StandardScaler(copy=True, with_mean=True, with_std=True)),
                        ('logisticregression',
                        LogisticRegression(C=1.0, class_weight=None, dual=False,
                                           fit_intercept=True, intercept_scaling=1,
                                           l1_ratio=None, max_iter=100,
                                           multi_class='multinomial', n_jobs=None,
                                           penalty='l2', random_state=123,
                                           solver='lbfgs', tol=0.0001, verbose=0,
                                           warm_start=False))],
                 verbose=False)
```

```
In [42]: clf_lr[1].coef_, clf_lr[1].intercept_
```



```
Out[42]: (array([[ -0.55597341,  0.57213952,  0.14144636,  0.73018788, -0.46457491,
                   0.5751959 , -0.175952 ,  0.21029227],
                  [ 1.25691352, -0.61626288, -0.16678387, -1.30535756,  0.21177478,
                  -0.97403424,  0.29214273, -0.44814125],
                  [-0.70094011,  0.04412336,  0.02533751,  0.57516967,  0.25280013,
                   0.39883834, -0.11619073,  0.23784898]]),
          array([ 0.15546789, -0.58741748,  0.43194959]))
```

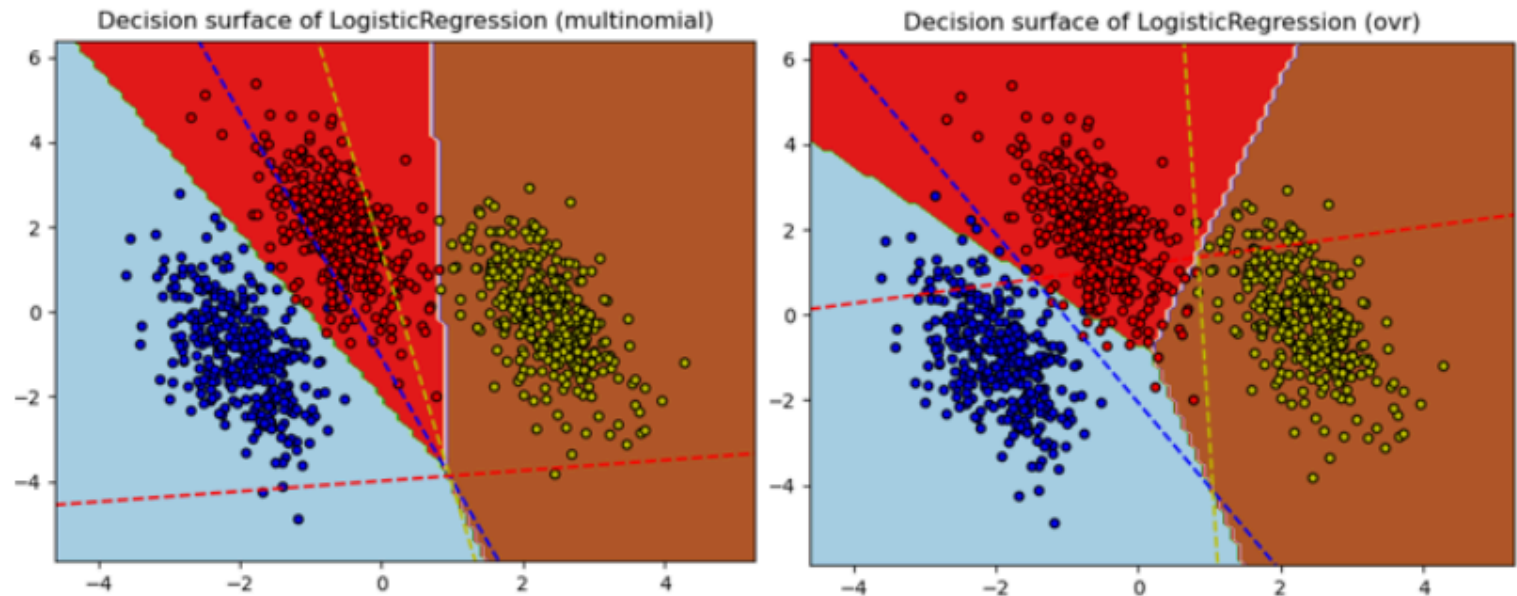
In [7]: Image('166.png')

Out[7]: **one-vs-rest(OVR)과 multinomial 비교**

- OVR에서는 각 평면들이 교차하는 영역에서의 부정확한 경계가

만들어질 수 있고, 이러한 문제는 클래스의 분포가 불균형할 때 심하게 나타납니다.

- 반면, OVR은 multinomial에 비해 직관적이며 해석이 용이합니다.



※ 이미지 출처: https://scikit-learn.org/stable/auto_examples/linear_model/plot_logistic_multinomial.html

선형 모델의 장단점

장점

- 계산 효율성
- 해석이 용이
- 이해하기 쉬운 개념

단점

- 대상 변수와 입력 변수간의 선형성을 지켜야 성능을 발휘
 - 성능 향상을 위한 많은 속성 엔지니어링이 필요합니다.
- 이상치에 민감

In []: